

ARCHON AGENTIC OS 2026 — LISTE COMPLÈTE DES FONCTIONNALITÉS

Catalogue système et modules, aperçu de l'architecture et matrice des fonctions

État : 11 novembre 2026 | Version : v0.5.1 (« Archon Hardened ») | Langue : Français

Catalogue système et modules, aperçu de l'architecture et matrice des fonctions

© Michael Keller

Table des matières

1. État des versions et aperçu des releases (v0.5.1) 3
2. Vues système et studios spécialisés (vues UI) 3
3. Écosystème d'économie de jetons et FinOps (minimisation des coûts) 8
4. Noyau, ordonnancement et architecture démons (pilier 1) 11
5. Modèle de processus multi-agents, hiérarchie et synthèse (pilier 2) 13
6. Pilotes MCP, services externes et outils multimodaux (pilier 3) 14
7. Mémoire, hygiène des bases de données et Obsidian 2nd Brain (pilier 4) 16
8. Sécurité, gouvernance Zero-Trust et diagnostic système (pilier 6) 18
9. Auto-évolution, marchés et auto-documentation (pilier 7) 19
10. Matrice de référence : plugins Evey/Hermes pilotables 20
11. Architecture d'entreprise et de résilience 22
12. Architecture d'autonomie, de bac à sable et de gouvernance 23
13. Cœur de performance, mise à l'échelle du débit et efficacité des jetons 25
14. Architecture moderne de mémoire et RAG 26
15. Exécution fiable et déterminisme (Durable Execution) 27
16. Sécurité, protection des données et garde-fous d'entreprise 29
17. Auto-optimisation, évaluation et intelligence en essaim 30
18. Pilier 3 : économie et ERP (alpha) et suite de mandants 31
19. Fonctionnalités d'architecture, de sécurité et de déploiement (v0.5.0 – v0.5.1) 32
20. Glossaire alphabétique et index 33

© Michael Keller

1. État des versions et aperçu des releases (v0.5.1)

Version	Nom de code	Jalons architecturaux et fonctionnels essentiels
v0.5.1	Archon Hardened	Routeurs FastAPI modulaires (main.py -88 %), 25 constats de sécurité critiques + 68 élevés corrigés, proxy d'admin Docker classe A, gestion d'utilisateurs RBAC (PBKDF2), mise à jour système en 2 phases (?local_only=true), intégration proxy AnythingLLM, installateur 1-Click portable.
v0.5.0	Économie & ERP (alpha)	Pilier 3 complet (sync CRM/PocketBase, articles/prestations, affaires ANG/AB, comptabilité avec DATEV et GiroCode, mandants Corporate), Research Lab v2 et 55 frameworks Boardroom.
v0.3.x	Sentinel / Foundation	Noyau OS à 7 piliers, cadre de 70 spécialistes, 34+ plugins Evey, routage FinOps à 3 niveaux, bacs à sable de code éphémères, débogage Replay, nettoyeur PII, cache sémantique et recherche hybride FTS5.

2. Vues système et studios spécialisés (vues UI)

1. Vue : Mission Control (cockpit de supervision et poste de commandement)

Problème : Sans cockpit central temps réel, la vue d'ensemble sur les agents actifs, le débit de jetons, les coûts journaliers et les validations bloquées fait défaut.

Mécanique : Regroupe 5 tuiles KPI vitales (charge de flotte, débit TPM, gardien des coûts, stabilité, portail d'approbation), la projection de panneau à 5 colonnes, la barre d'inspection des processus actifs (pied de page à 3 emplacements pour battement de cœur, concurrence d'outils et saturation de contexte), le ruban d'activité de jetons de 15 minutes (sparkline SVG) et le flux d'audit en direct.

Bénéfice : *Fournit une supervision 360° sans délai de l'ensemble du système d'exploitation sur un seul écran.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

2. Vue : Operations – tableau Kanban et moteur de tâches

Problème : Les tâches se perdent dans de purs fils de chat et ne peuvent être pilotées ni par statut ni par responsabilités.

Mécanique : 5 colonnes de cycle de vie (To-Do, En cours, Review & QA, Bloqué, Terminé) directement sur kanban.db (source unique de vérité). Prend en charge les limites WIP configurables par colonne, les filtres de priorité (P0–P3), l'affectation d'agents, le suivi du temps et la liaison Obsidian en 1 clic. Intègre le portail qualité à 4 yeux : avant de déplacer une tâche vers « done », un pair réviseur (Katharina Völk pour QM/Conformité ou Robert) contrôle le résultat sur l'exhaustivité, la substance et la capacité d'action (score 1–100 %). En cas de « changes_requested », un verrou de statut s'applique ; une dérogation Human-in-the-Loop (force=True) permet de passer outre au besoin.

Bénéfice : *Structure tous les projets opérationnels en transparence et évite la surcharge du système par des garde-fous de travaux en cours.* Empêche les états d'avancement inachevés au statut « done » grâce à une revue à quatre yeux obligatoire.

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

3. Vue : Operations – organigramme 2D interactif et lignes de reporting

Problème : Des listes plates d'agents ne montrent ni pouvoirs de direction ni voies d'escalade en cas de processus bloqués.

Mécanique : Visualise 56 nœuds et 55 lignes de reporting sous forme de graphe 2D interactif (Apex -> CEO Robert -> responsables spécialisés -> spécialistes). Prend en charge le glisser-déposer pour rattacher les

supérieurs, l'alignement sur grille, la vue cartes d'équipe et l'enregistrement durable dans config/org_hierarchy.json.

Bénéfice : *Définit des chaînes de délégation et d'escalade contraignantes que le système suit automatiquement en cas de blocage.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

4. Vue : Operations – communication interne des agents (trafic radio)

Problème : Quand les spécialistes se передают des sous-tâches entre eux, cela inonde soit le chat utilisateur, soit reste invisible.

Mécanique : Centrale d'écoute dédiée avec liste de canaux (#général, #tech-dev, #juridique, #marketing) et fil de messages chronologique de toutes les concertations inter-agents issues de kanban.db.

Bénéfice : *Rend la collaboration de la flotte d'agents retraçable sans charger la boîte principale de l'utilisateur.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

5. Vue : Entrée & triage (portier HITL à 3 colonnes et boîte aux lettres)

Problème : Les actions d'écriture incontrôlées des agents (p. ex. envoi d'e-mails) ainsi que les entrées dispersées du courrier et de Telegram comportent des risques de sécurité.

Mécanique : Centrale de triage à trois colonnes : colonne 1 pour les validations Human-in-the-Loop surlignées en jaune (Valider/Refuser), colonne 2 pour la synchronisation e-mail IMAP/SMTP avec tri automatique et délégation, colonne 3 pour le flux Telegram en direct reflété (@Beatman_bot).

Bénéfice : *Garantit qu'aucune opération d'écriture externe n'est exécutée sans validation expresse de l'opérateur.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

6. Vue : cadre des agents et gestion de flotte (cockpit à 8 onglets)
Problème : les spécialistes ont besoin de modèles individuels, de prompts système, d'outils et d'instructions de travail en un lieu central.
Mécanique : gère 70 profils de spécialistes avec filtres rapides (Actif, Favoris, Tous, 7 domaines), permutation à chaud du fournisseur et du modèle ainsi que 8 onglets spécialisés : aperçu (dossier 360° et nom d'appel Telegram), chat direct, compétences (Skills), opérations (historique des tâches), SOUL (éditeur de prompt système), dossier personnel (SOP), marché des talents et marché des Skills.
Bénéfice : service du personnel central pour calibrer et étendre en quelques secondes l'ensemble du socle des agents.
Prix et revers (arbitrage) : intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

7. Vue : Master Chat (espace de travail double mode et barre latérale multi-sessions)

Problème : Pour travailler directement avec les agents, il manquait un environnement plein écran ciblé permettant de basculer rapidement entre dialogue 1:1, chat de groupe et ordre de mission.

Mécanique : Offre le commutateur double mode (« Direct » pour réponse immédiate vs « Mission » pour création de tâche structurée), le téléversement multi-fichiers (jusqu'à 5 fichiers), la capture d'écran du navigateur (bouton caméra), la saisie vocale (dictée), le sélecteur de budget de raisonnement (0 à 16k jetons), l'optimiseur de prompts IA et l'historique de sessions à droite (agents vs groupe). Inclut le pipeline vocal Telegram : les messages vocaux entrants (.ogg/Opus) en déplacement sont transcrits via Whisper/Gemini,

routés selon les noms d'appel (/robert, /anwalt, etc.) vers l'agent spécialisé, et les réponses sont renvoyées directement sur Telegram via edge-tts (de-DE-ConradNeural) sous forme de mémos vocaux naturels.

Bénéfice : *Concilie dialogue à haute vitesse, saisie multimodale et pilotage de missions sur une surface de travail continue.* Permet un pilotage vocal mobile complet et un échange audio bidirectionnel via Telegram.

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

8. Vue : AI Mastermind & Boardroom (55 frameworks méthodologiques)

Problème : les décisions stratégiques complexes exigent des méthodes de gestion structurées et des perspectives controversées plutôt que des avis isolés.

Mécanique : simule des séances de comité multi-niveaux (Chairman + 3 emplacements réels d'agents spécialisés + challenger Red-Team) à partir de 34 frameworks sélectionnables en 7 catégories (35 de base) (p. ex. CLEAR, Design Sprint, Double Diamond, Lean Startup, First Principles, Théorie des jeux, Blue Ocean, Pre-Mortem, Six Sigma).

Bénéfice : impose la rigueur méthodique, révèle les angles morts grâce à la Red-Team et produit des mémos de conseil prêts à décider.

Prix et revers (arbitrage) : intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

9. Vue : Showroom (explorateur de projets, galerie d'artefacts et bac à sable Python)

Problème : Les rapports générés, prototypes HTML, chiffrages et scripts se perdent dans de longues conversations.

Mécanique : Catalogue tous les résultats de travail par créateur, date et type. Inclut le lecteur de coffre Workspace avec rendu en direct pour Markdown/HTML/JSON ainsi qu'un bac à sable Python isolé pour une exécution sûre du code. Étendu avec le Live-Runner interactif du Showroom : permet l'exécution directe de fichiers Python et de code dans le navigateur avec timeout de 12 s, répertoire de travail isolé et tiroir de terminal dépliable (STDOUT, STDERR, code de sortie et durée en millisecondes).

Bénéfice : *Sert de hall de réception et de présentation pour tous les livrables finis avant usage client ou production.* Les artefacts de code et scripts peuvent être vérifiés et testés directement dans le Showroom sans changer de console.

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

10. Vue : bibliothèque centrale et 2nd Brain (RAG, Auto-OCR et Obsidian-Vault)

Problème : livres spécialisés, normes DIN et plans d'aménagement en PDF ne pouvaient jusqu'ici être fouillés par les agents sans copie manuelle.

Mécanique : réunit 3 sous-onglets et 4 hubs centraux (00_HR_Agenten, 02_Wissensbibliothek, Protokolle/, 00_Notebook_Synthesen) : 1. index RAG AnythingLLM (port 3001), 2. capteur de documents avec glisser-déposer, Auto-OCR (Docling) et classification thématique IA, 3. arborescence Obsidian directe et visionneuse Markdown avec commutateur de coffre.

Bénéfice : transforme les expertises PDF en connaissances Markdown et vectorielles interrogeables pour les 70 agents, de façon entièrement automatique.

Prix et revers (arbitrage) : intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

11. Vue : moteur de workflow (maille d'automatisation n8n sur le port 5678)

Problème : Les processus d'interface récurrents (tâches Cron, webhooks, rapprochements de bases) consomment inutilement des jetons quand ils tournent via des LLM.

Mécanique : Intègre l'éditeur visuel de workflow n8n et le relie via la compétence n8n-automation-mesh (FastMCP) à l'agent CEO Robert. Routines 0-jeton activées : 1. briefing matinal (jours ouvrés 07h30 avec dépôt Obsidian et remise Telegram), 2. maintenance hebdomadaire (dimanches 23h00 avec sauvegarde SQLite et audit de santé), 3. chien de garde RSS et actualités (scrutation horaire), plus 5 webhooks fonctionnels actifs pour extraction web, messagerie, rendu documentaire, recherche de flux et sync CRM.

Bénéfice : *Combine des processus de fond déterministes à 0 jeton avec des décisions d'agents cognitives aux points clés.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

12. Vue : passerelle LLM (tableau de bord du routeur IA OmniRoute)

Problème : L'attachement fixe à un seul fournisseur cloud entraîne un arrêt immédiat du système en cas de limites de débit ou de pannes.

Mécanique : Tableau de bord proxy central (port 20128 / 8000) avec coffre multi-fournisseurs, mesure de latence temps réel (ms), moniteur d'erreurs HTTP (429/500), coupe-circuit automatique et cascade de repli à 4 niveaux (GLM-4.7 Flash -> Gemini 2.5 Flash -> Claude 3.5 Sonnet -> Llama 3.3 70B local).

Bénéfice : *Assure 99,9 % de disponibilité de tous les processus cognitifs à coûts API minimaux.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

13. Vue : Token Guard (Guardian et analytique de jetons en 3 sous-vues)

Problème : Manque de transparence sur quel agent, quel modèle ou quelle session isolée cause le plus de jetons et de coûts.

Mécanique : Offre des filtres temporels (24h, 7j, 30j, total), un minuteur d'inactivité et 3 sous-vues : 1. aperçu et graphique de tendance journalière avec tableaux de profils/modèles et formulaire de garde-fous, 2. inspecteur de consommation par session isolée (jetons d'entrée, de sortie, de lecture de cache triables), 3. ressources et limitation des tâches (curseurs 1–16 tâches).

Bénéfice : *Contrôle total des coûts jusqu'à l'ID de session isolée, y compris limitation immédiate du budget et du parallélisme.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

14. Vue : extensions et centrale de modules (34 plugins spéciaux Evey)

Problème : Les fonctions de sécurité, de mémoire et d'analyse ne pouvaient jusqu'ici être activées ou désactivées de façon modulaire par domaine spécialisé.

Mécanique : Pilote 34 plugins Hermes/Evey spécialisés via 8 onglets spécialisés : sécurité et gardiens (4), performance et cache (2), supervision et analytique (3), intelligence des agents (6), planification et tâches (2), ponts et interfaces (4), mémoire et connaissances (3), outils et exécution (3) plus finances/qualité/maintenance. Inclut le système de plugins standardisé et les crochets runtime : arborescence modulaire (plugins/<plugin_id>/plugin.json), 4 crochets de cycle de vie (on_load, on_enable, on_disable, on_unload) et 2 intercepteurs d'outils (before_tool_execute, after_tool_execute) avec plugins de démarrage préinstallés (System Metrics, Security Audit Hook, Cost Guard).

Bénéfice : *Permet d'activer et de désactiver précisément des capacités système isolées en cours d'exploitation par simple clic de souris.* Permet des extensions runtime isolées et une interception globale des outils sans toucher au code du noyau.

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

15. Vue : Research Lab (recherche profonde et triangulation des preuves)

Problème : Les requêtes simples livrent des résultats isolés non vérifiés, sans recoupement des sources ni évaluation de confiance.

Mécanique : Cockpit de recherche multi-niveaux avec profondeur de recherche réglable, moniteur de sources, score de preuve, journal de synthèse en direct pour trianguler les affirmations contradictoires et export en 1 clic vers l'Obsidian-Vault. Inclut la boucle autonome de recherche profonde et de réflexion v2 (boucle Re-Search) : décomposition des recherches complexes en 3–5 sous-questions, analyse autonome des lacunes, jusqu'à 3 cycles de recherche successifs itératifs, repli multi-fournisseurs (SearXNG / Jina Reader) et protection SSRF.

Bénéfice : *Produit des dossiers de marché, juridiques et techniques solides et citables.* Comble les lacunes de connaissances de façon autonome par des recherches successives récursives en cas de sources contradictoires ou incomplètes.

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

16. Vue : télémétrie et journaux (flux d'audit en direct à 3 sources)

Problème : En cas d'erreurs dans les conteneurs ou les prompts d'agents, il fallait jusqu'ici chercher laborieusement dans le terminal via des commandes Docker.

Mécanique : Regroupe 3 sources de journaux en temps réel : journaux physiques des conteneurs Docker (stdout/stderr des 11 conteneurs), journaux cognitifs des agents et piste d'audit cryptographique. Avec filtre de niveau (DEBUG, INFO, WARN, ERROR), verrou de défilement auto et export.

Bénéfice : *Enregistreur de vol infalsifiable pour un diagnostic d'erreurs en quelques secondes et des preuves de conformité.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

17. Studio spécialisé : Archon Prompt Studio (832 modèles et SOP Fabric)

Problème : des tâches formulées avec imprécision entraînent hallucinations, formats de sortie erronés et coûteuses boucles de correction.

Mécanique : kit intégré avec 824 modèles d'entreprise en 16 catégories spécialisées, plus de 255 motifs SOP Fabric et la formule à 5 champs (1. rôle, 2. contexte, 3. tâche, 4. format, 5. contraintes/règles négatives) ainsi qu'un optimiseur de prompts IA.

Bénéfice : contraint chaque délégation dans une structure limpide et la transfère en un clic vers le chat ou une tâche Kanban.

Prix et revers (arbitrage) : intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

18. Studio spécialisé : Visual CSS Live Studio et gizmo de transformation in-place (Alt + S)

Problème : Les adaptations de design exigeaient jusqu'ici de fouiller dans les fichiers CSS et de recharger sans cesse le navigateur.

Mécanique : Studio de design flottant temps réel (Alt + S) avec inspecteur DOM en croix, gizmo de transformation interactif (tirer la position, redimensionner 4 coins/bords, tirer l'arrondi des coins), gestionnaire de profils de thèmes et export CSS direct vers custom_studio.css.

Bénéfice : *Permet un stylage en direct au pixel près de toute l'interface en cours d'exploitation, sans rechargement de page.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

19. Studio spécialisé : Design System Lab, architecte AppShell et studio à 15 composants

Problème : Des hauteurs d'en-tête, espacements ou contrastes divergents entre mode clair et mode sombre troublent le calme visuel.

Mécanique : Inclut l'architecte AppShell interactif (alignement d'en-tête 56px, largeur de barre latérale, marges de viewport avec mini-simulation en direct) ainsi que le studio de design à 15 composants (os-workspace à os-badge) avec moteur double thème (Light/Dark).

Bénéfice : *Garantit une apparence B2B-SaaS uniforme et sans éblouissement sur les 20 vues.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

20. Vues spéciales : System Control Center (11 onglets spécialisés, dock en direct) et Omnibox globale (Ctrl + K)
Problème : les réglages étaient dispersés ; la navigation entre 20 vues et 70 agents exigeait de nombreux clics de souris.
Mécanique : le System Control Center regroupe 7 onglets d'administration (Hermes Core avec éditeur YAML, conteneurs et services, fournisseurs et passerelle, méthodologies, plugins, communication, design) et peut s'ancrer à droite via le « Live-Dock ». L'Omnibox (Ctrl + K) navigue au clavier instantanément vers chaque onglet (#), agent (@) ou commande. Intègre le système de mise à jour en 1 clic et le déploiement continu : sync GitHub automatique avec instantanés pré-update de toutes les bases SQLite (WAL) dans data/snapshots/ avant chaque Git-Pull, rollback en 1 clic et contrôles Git-Stat minimaux contre les timeouts de montage.
Bénéfice : vitesse de manipulation maximale pour administrateurs et utilisateurs avancés. Permet des mises à jour logicielles sans danger avec rollback automatique de base de données en cas d'erreur.
Prix et revers (arbitrage) : intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

3. Écosystème d'économie de jetons et FinOps (minimisation des coûts)

21. Plafonds de routage de coûts automatiques à 3 niveaux (LOW / MEDIUM / HIGH)

Problème : Quand de simples tâches de routine tournent sur des modèles haut de gamme coûteux (comme Claude Sonnet ou GPT-4o), les coûts journaliers explosent.

Mécanique : Plafonds de prix configurables par million de jetons dans le System Control Center : LOW (max. 0,10 $/1M -> impose le niveau gratuit/Flash-Lite), MEDIUM (max. 1,00 $/1M -> Haiku/Gemini Flash), HIGH (sans plafond -> meilleurs modèles de raisonnement pour architectures complexes).

Bénéfice : *Réduit les dépenses API courantes au quotidien jusqu'à 85 %, sans perdre en qualité sur les tâches difficiles.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

22. Proxy Docker FreeLLM-API et routage « auto/best-free » (port 3002)

Problème : Les contingents gratuits des différents fournisseurs sont difficiles à surveiller manuellement et changent souvent.

Mécanique : Un conteneur Docker dédié (freellmapi sur le port 3002) regroupe des points de terminaison LLM gratuits. Le profil de routage « auto/best-free » choisit automatiquement le modèle à zéro euro le plus puissant disponible.

Bénéfice : *Permet l'exploitation continue des agents de fond et de triage à exactement 0,00 $ de coûts API.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

23. Pilotage continu du budget de raisonnement dans le Master Chat et le routeur (0 à 16k jetons)

Problème : Les modèles de raisonnement (mode Thinking) consomment souvent des milliers de jetons de réflexion invisibles pour des questions triviales.

Mécanique : Curseur à 6 niveaux (Off : 0 tok, Minimal : ~1k, Bas : ~2k, Moyen : ~4k, Haut : ~8k, Maximal : ~16k) dans le pied de page du chat et dans agent_router.py.

Bénéfice : *Coupe complètement les coûteuses boucles de réflexion pour les tâches simples et réserve le raisonnement profond aux vrais cas complexes.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

24. Cache Evey et pilotage TTL du cache (cache de réponses sémantique et exact de 24h)

Problème : Les requêtes système récurrentes identiques ou très similaires entraînaient à chaque fois de nouveaux appels API payants.

Mécanique : Le plugin evey-cache stocke les réponses des modèles avec une durée de vie configurable (standard : TTL de 24 heures) et affiche séparément dans l'inspecteur de session les jetons de lecture de cache économisés.

Bénéfice : *Répond aux requêtes répétées en 0 milliseconde à 0 jeton de coût.*

Prix et revers (arbitrage) : Si la TTL du cache est trop élevée (> 72h), le système fournit des réponses périmées sur les sujets dynamiques.

Quand désactiver ? et seuils : Désactiver pour les requêtes temps réel (cours en direct, état des serveurs). TTL std : 24h.

25. Écrêteurs durs Max-Context et Max-History dans le Guardian (32k / 20 tours)

Problème : Des sessions de chat croissant sans fin renvoient à chaque nouvelle question tout l'historique et font grimper les jetons d'entrée de façon exponentielle.

Mécanique : Garde-fous réglables dans le Guardian pour la fenêtre de contexte maximale (p. ex. 32 000 jetons) et le nombre maximal de messages transmis (p. ex. 20 derniers tours).

Bénéfice : *Plafonne de façon fiable vers le haut les coûts d'entrée maximaux possibles par tour.*

Prix et revers (arbitrage) : Tronque durement les messages au-delà de 32k jetons ou 20 tours, ce qui fait disparaître le contexte le plus ancien.

Quand désactiver ? et seuils : Pour les analyses documentaires profondes, relever à 64k–128k dans Hermes Core.

26. Écrêteur dynamique de jetons pour modèles à faible contexte (backend/token_clipper.py)

Problème : Les modèles compacts ou locaux (8k/16k) décrochent quand de volumineuses sorties d'outils font éclater la fenêtre de contexte.

Mécanique : Préserve impérativement le prompt système (index 0) et la dernière instruction utilisateur, raccourcit intelligemment les blocs isolés trop longs et supprime les étapes intermédiaires anciennes.

Bénéfice : *Permet l'usage stable de tout petits modèles extrêmement bon marché même pour des tâches volumineuses.*

Prix et revers (arbitrage) : Raccourcit les sorties d'outils trop longues par le milieu ([...SNIP...]), si bien que des lignes de code isolées peuvent manquer dans le texte brut.

Quand désactiver ? et seuils : Désactiver ponctuellement quand des journaux bruts complets non tronqués sont requis.

27. Compression mémoire à fenêtre glissante (contexte de décision consolidé)

Problème : En tronquant simplement les anciens messages, les décisions de projet antérieures se perdent.

Mécanique : Condense les anciens tours de conversation en un résumé compact et l'injecte comme bloc [KONSOLIDIERTER GESPRÄCHS- & ENTSCHEIDUNGSKONTEXT] en position 1.

Bénéfice : *Économise jusqu'à 70 % des jetons de conversation historiques tout en conservant pleinement toutes les décisions prises.*

Prix et revers (arbitrage) : L'étape de compression coûte ponctuellement ~800 jetons et lisse les formulations

littérales des tours antérieurs.

Quand désactiver ? et seuils : Ajuster au besoin via protect_last_n (std : 20) dans Hermes Core.

28. Pagination LRU-K de la RAM des sous-agents (backend/context_swapper.py)

Problème : Des sous-agents parallèles maintiennent simultanément d'énormes volumes de données brutes dans la mémoire active des prompts.

Mécanique : Décharge les pages de contexte qui ne sont plus activement requises dès 80 % d'utilisation selon le principe du moins récemment utilisé vers SQLite et ne les recharge qu'au besoin (réhydratation).

Bénéfice : *Maintient les prompts allégés et évite des frais de jetons inutiles pour des connaissances de fond inactives.*

Prix et revers (arbitrage) : La décharge et le rechargement de la RAM d'agents vers le disque génèrent ~10–25 ms de latence I/O lors du changement de contexte.

Quand désactiver ? et seuils : Laisser actif dès 80 % d'utilisation RAM pour éviter les plantages OOM.

29. Routeur JEV — préfiltration d'intention déterministe à 0 jeton (jev_router.py)

Problème : Classifier chaque simple instruction système via un grand modèle de langage gaspille jetons et temps.

Mécanique : Classifie les entrées arrivantes avant l'appel LLM via un vecteur d'exécution JSON déterministe et une heuristique de mots-clés.

Bénéfice : *Achemine les ordres système clairs directement vers la routine cible sans consommer un seul jeton LLM.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

30. Prompteur ASD-STE100 Simplified Technical English (.agents/skills/asd-ste100-prompter)

Problème : Les proses verbeuses dans les prompts système et instructions d'agents gonflent inutilement chaque appel API.

Mécanique : Transforme les instructions selon la norme aéronautique ASD-STE100 (un sens par mot, verbes actifs, phrases courtes, zéro remplissage).

Bénéfice : *Réduit la longueur en jetons des prompts système d'environ 30–40 % et abaisse simultanément le taux d'erreur des appels d'outils.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

31. Anti-AI-Slop et compression de sortie (.agents/skills/anti-ai-slop)

Problème : Les modèles de langage produisent par défaut de longues formules de politesse, introductions et résumés redondants (coûteux jetons de sortie).

Mécanique : Bannit les tics IA, phrases creuses et gonflement artificiel directement dans le prompt système et nettoie les sorties.

Bénéfice : *Économise jusqu'à 25 % des jetons de sortie particulièrement coûteux et livre des textes spécialisés immédiatement exploitables.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

32. Estimateur de coûts pré-vol et prévision budgétaire (backend/cost_estimator.py)

Problème : Avant de lancer de grandes tâches, il était impossible de prévoir ce que l'exécution coûterait.

Mécanique : Calcule au préalable via POST /api/router/estimate-cost les jetons d'entrée/sortie attendus ainsi que le montant en dollars d'après la grille tarifaire actuelle des modèles et le compare au budget de tâche.

Bénéfice : *Avertit des exécutions coûteuses avant même que le premier centime ne soit dépensé.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active

d'actions LLM au sein du module).

33. Budget de tâche dynamique avec rétrogradation économe à 80 % et arrêt à 100 %

Problème : Une seule tâche complexe pouvait engloutir inaperçu tout le budget journalier.

Mécanique : Surveille les coûts cumulés par tâche (p. ex. limite 0,25 $) : bascule automatiquement à 80 % vers un modèle économe bon marché et met la tâche en pause à 100 % avec demande de validation Telegram.

Bénéfice : *Prévient les dérapages de coûts lors de recherches autonomes multi-niveaux.*

Prix et revers (arbitrage) : À 80 % du budget de tâche, bascule vers un modèle économe moins cher ; à 100 %, arrêt dur.

Quand désactiver ? et seuils : Relever le budget de tâche (std : 0,50 $) à 2,00 $ pour les grandes exécutions d'architecture.

34. Tueur de boucles et limitation Max-Steps (15 étapes / Jaccard > 90 %)

Problème : Quand un agent s'engage dans une boucle infinie sur un appel d'outil défectueux, il brûle du crédit à la seconde.

Mécanique : Contrôle chaque étape pour répétitions exactes (>= 3) ou références circulaires sémantiques (> 90 % de similarité) et plafonne les tâches au plus tard après la limite Max-Steps configurable (standard : 15 étapes).

Bénéfice : *Protège le compte de façon fiable contre les boucles infinies inaperçues pendant la nuit.*

Prix et revers (arbitrage) : Des seuils trop stricts (Max identique = 2 ou Jaccard < 0,80) arrêtent trop tôt des tâches itératives légitimes comme le test-fixing progressif ou la pagination.

Quand désactiver ? et seuils : Désactiver ponctuellement quand un agent doit traiter sciemment de nombreuses étapes de boucle uniformes (p. ex. migration massive de 50 fichiers). | Seuils : Max identique : 2 = stoppe les réessais | > 5 = boucles coûteuses (std : 3). Jaccard : < 0,80 = fausses alertes | > 0,96 = détecte les boucles tard (std : 0,90). Max étapes : std 15.

4. Noyau, ordonnancement et architecture démons (pilier 1)

35. Orchestrateur maître Hermes (Hermes-Core)
Problème : des agents isolés non coordonnés travaillent les uns contre les autres ou perdent de vue l'objectif global.
Mécanique : centrale de pilotage des processus pour l'analyse des ordres, le choix des rôles, la supervision et la réception des résultats.
Bénéfice : assure la cohérence stratégique entre les 70 spécialistes.
Prix et revers (arbitrage) : davantage de services de fond actifs et un raisonnement élevé (« high ») augmentent la charge de base et les coûts en jetons.
Quand désactiver ? et seuils : pilotable de façon granulaire via les 14 sections de l'onglet « HERMES CORE ».

36. Démon noyau sans tick et boucle d'événements Asyncio (port 8420 / 8501)

Problème : L'interrogation permanente à la seconde gaspille les ressources CPU et l'énergie système.

Mécanique : Calcule le moment exact de la prochaine tâche due (« Sleep-until-next-due ») et met le noyau en veille jusque-là.

Bénéfice : *Charge système minimale au repos avec réaction de réveil sans délai.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

37. Ordonnanceur à préemption par priorités (file P0–P3)

Problème : Les questions urgentes des utilisateurs en chat direct devaient attendre quand les tâches de fond occupaient tous les créneaux.

Mécanique : File priorisée : les tâches P3 (chat direct / urgence) doublent immédiatement les routines P1/P2 en

cours.

Bénéfice : *Garantit des temps de réponse rapides en dialogue malgré une forte charge de fond.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

38. Limitation de concurrence librement évolutive (backend/concurrency_throttle.py)

Problème : Trop d'agents simultanés entraînent des limites de débit HTTP-429 et des pics CPU à 99 %.

Mécanique : Régule les workers actifs via un sémaphore Asyncio (réglable par curseur de 1 à 16 tâches parallèles dans le Guardian et le Control Center).

Bénéfice : *Permet un fonctionnement protégé et bridé sur laptops aussi bien qu'une exploitation à forte charge sur stations de travail.*

Prix et revers (arbitrage) : Des limites de stagnation trop justes (< 30 s) interrompent à tort les lents modèles de raisonnement profond (p. ex. o3/R1) en plein processus de réflexion ; plus de workers augmentent les pics RAM et API.

Quand désactiver ? et seuils : Désactiver pour les analyses isolées très longues ou les exécutions LLM locales sur matériel lent. | Seuils : Max workers : 1–2 = arriéré | > 8 = pics RAM et HTTP-429 (std : 4). Stagnation : < 30s = interrompt les penseurs | > 180s = guérison tardive (std : 45s).

39. Chien de garde autonome à 30 secondes et reprise sur crash (watchdog_daemon.py)

Problème : Les tâches plantées ou gelées restaient durablement bloquées au statut in_progress.

Mécanique : Contrôle toutes les 30 secondes l'ensemble des processus en cours, abandonne les tâches zombies après timeout (120s) et transfère les tâches orphelines vers la reprise automatique sur crash.

Bénéfice : *Maintient tous les créneaux de workers libres de dossiers morts.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

40. Superviseur de deadlocks et de stagnation (backend/deadlock_supervisor.py)

Problème : Des dépendances mutuelles entre deux agents peuvent conduire à un gel silencieux.

Mécanique : Détecte les états d'attente circulaires et les tâches sans progrès (> 45s) et lève le blocage de façon autonome avec backoff exponentiel.

Bénéfice : *Prévient les arrêts de pipeline lors de travaux de groupe complexes.*

Prix et revers (arbitrage) : Une limite de stagnation trop juste (< 30s) interrompt les lents modèles de raisonnement profond (o3/R1) en plein processus de réflexion.

Quand désactiver ? et seuils : En cas d'usage de modèles de raisonnement, relever la limite de stagnation à 90–120s (std : 45s).

41. Auto-guérison de flotte et redémarrage auto des conteneurs (backend/fleet_self_healing.py)

Problème : Quand l'un des 11 conteneurs Docker tombait, il fallait le redémarrer manuellement.

Mécanique : Surveille tous les conteneurs centraux via le docker-socket-proxy sécurisé et effectue en cas de panne (> 60s) un redémarrage autonome avec entrée d'audit.

Bénéfice : *Assure l'exploitation 24/7 sans surveillance de tout le paysage Docker.*

Prix et revers (arbitrage) : Un conteneur durablement défectueux peut entrer en boucle de redémarrage via l'auto-restart.

Quand désactiver ? et seuils : Désactiver ponctuellement pendant les travaux manuels de maintenance des conteneurs.

42. Standup matinal proactif et service de réveil (backend/autonomous_standup.py)

Problème : Le matin, il manquait un aperçu compact des exécutions nocturnes, des coûts et des décisions en attente.

Mécanique : Génère chaque jour à 08h00 un rapport de statut consolidé dans l'Obsidian-Vault (01_PROJEKTE/Daily_Briefings/) et le pousse vers le tableau de bord et Telegram.

Bénéfice : *Démarre chaque journée de travail avec un dossier de décision prêt.*

Prix et revers (arbitrage) : Chaque briefing matinal automatique consomme ~1 500–3 000 jetons au moment réglé.

Quand désactiver ? et seuils : Désactiver si aucun rapport matinal quotidien n'est souhaité (std : 08h00).

5. Modèle de processus multi-agents, hiérarchie et synthèse (pilier 2)

43. 70 profils d'agents déclaratifs et dossiers personnels Markdown
Problème : les assistants IA génériques n'ont ni conscience de rôle ni normes de qualité définies.
Mécanique : chacun des 70 spécialistes est ancré dans agent_profiles.json ainsi que dans un dossier personnel propre (00_HR_Agenten/) avec prompt système, route de modèle, périmètre d'outils et SOP.
Bénéfice : crée des collaborateurs virtuels fiables pour la stratégie, le droit, le bâtiment, les finances, le marketing et l'informatique.
Prix et revers (arbitrage) : intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

44. Délégation Robert, synchronisation fan-in et contrat de marqueur Lisa (SYNTHESE-BEREIT)

Problème : Avec des sous-tâches parallèles, des résultats partiels se perdaient ou étaient résumés de façon incomplète.

Mécanique : Robert décompose les grands projets ; un verrou de processus (_MC_DATA_LOCK) attend l'achèvement de toutes les sous-tâches (fan-in). Lisa agrège les rapports en contrôlant le marqueur obligatoire SYNTHESE-BEREIT ; Robert donne en ligne 1 la VALIDATION formelle.

Bénéfice : *Assurance qualité multi-niveaux sur le modèle d'une vraie direction d'entreprise.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

45. Bureau de transfert standardisé d'agent à agent (backend/handoff_desk.py)

Problème : Pertes de contexte silencieuses lors du passage d'un spécialiste au suivant.

Mécanique : Génère des procès-verbaux formels de transfert avec livrables, hypothèses ouvertes et somme de contrôle SHA-256, qui doivent être quittancés par l'agent destinataire.

Bénéfice : *Interfaces révisables entre tous les services.*

Prix et revers (arbitrage) : Des contrôles SLA stricts rejettent des réponses factuelles très courtes mais correctes quand min_content_length est trop élevé (> 300).

Quand désactiver ? et seuils : Standard : 100 caractères de longueur minimale, 60 min de délai.

46. Ordonnanceur DAG à deux niveaux pour exécution parallèle de projets (backend/dag_scheduler.py)

Problème : Le traitement séquentiel de nombreuses sous-tâches prend trop longtemps sur les grands projets.

Mécanique : Calcule le graphe acyclique dirigé (DAG) : exécute les recherches indépendantes (niveau 1) en parallèle puis lance automatiquement la synthèse (niveau 2).

Bénéfice : *Raccourcit le délai d'exécution des analyses complexes d'un multiple.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

47. Auto-correction et boucle de réflexion d'exécution d'outils

Problème : Des erreurs fugaces de syntaxe ou de paramètres dans les appels d'outils entraînaient l'abandon

immédiat.

Mécanique : Intercepte les erreurs d'outils, réfléchit au message d'erreur et corrige les paramètres d'appel de façon autonome en jusqu'à 3 tentatives.

Bénéfice : *Robustesse nettement accrue pour les opérations fichiers, bases de données et web.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

48. Workflows longue durée interruptibles avec instantanés d'état SQLite

Problème : Quand une tâche attend des heures une validation utilisateur, elle occupe inutilement de la mémoire ou perd son état lors d'un redémarrage.

Mécanique : Fige l'état d'exécution exact dans la table workflow_checkpoints et reprend le travail sans rupture après validation.

Bénéfice : *Permet des processus d'affaires de plusieurs jours au-delà des redémarrages.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

49. Portail de consensus à deux agents et revue critique adverse (backend/consensus_gate.py)

Problème : Un agent isolé ne remarque souvent pas ses propres erreurs de raisonnement ou hallucinations sur les tâches critiques.

Mécanique : Impose pour les tâches P3 et les interventions critiques une relecture formelle par un agent réviseur indépendant (Red-Team/Critic). Câblé dans le tableau Kanban comme portail qualité automatique à 4 yeux (revue Katharina Völk / Robert avec score 1–100 % et verrou de statut en cas de besoin de révision).

Bénéfice : *Principe des quatre yeux pour tous les résultats critiques pour l'entreprise.*

Prix et revers (arbitrage) : Triple (3x) la consommation de jetons et augmente le temps de réponse de +6 à +20 s ; l'unanimité comporte un risque de deadlock.

Quand désactiver ? et seuils : N'activer qu'à partir de la priorité P3/P4 et choisir la majorité aux 2/3.

6. Pilotes MCP, services externes et outils multimodaux (pilier 3)

50. Écosystème Archon CRM et PocketBase (ports 3005, 8090, 8092)

Problème : Contacts clients, historiques e-mail et assets médias se trouvaient séparés des agents IA.

Mécanique : Frontal CRM dédié (port 3005) avec backend PocketBase (port 8090) et serveur média (port 8092), entièrement connecté via scripts/archon_crm_mcp.py.

Bénéfice : *Les agents peuvent récupérer des contacts, les approfondir, les noter (lead scoring) et y affecter la correspondance.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

51. Appariement automatique d'e-mails et importateur d'archives LinkedIn

Problème : Des milliers de contacts issus d'exports LinkedIn et de boîtes e-mail devaient être rapprochés manuellement.

Mécanique : Synchronise les boîtes aux lettres (--fetch-mails), affecte automatiquement les e-mails aux contacts CRM existants et importe des CSV complets de connexions LinkedIn avec déduplication.

Bénéfice : *Construit en quelques minutes une base relationnelle complète et enrichie.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active

d'actions LLM au sein du module).

52. Gestionnaire d'autorisations d'outils Robert (backend/tool_permission_manager.py)

Problème : Les agents ne doivent pas abandonner démunis quand des paquets Python/Node manquent, mais ne doivent rien installer non plus sans qu'on le leur demande.

Mécanique : Robert dépose une demande d'installation formelle (pending_permission) avec commande et justification ; ce n'est qu'après ta validation en 1 clic que le paquet est installé.

Bénéfice : *Extensibilité contrôlée avec pleine protection du système hôte.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

53. Maille webhook événementielle (backend/event_mesh.py)

Problème : Les événements externes de GitHub, formulaires ou systèmes tiers ne pouvaient déclencher aucune action d'agent.

Mécanique : Récepteur webhook universel qui valide les payloads JSON externes et les convertit directement en tâches Kanban exécutables.

Bénéfice : *Relie Archon OS en temps réel au reste du paysage informatique.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

54. Hub central de notifications multi-canal (backend/notification_hub.py)

Problème : Les alertes importantes se perdaient dans l'UI tandis que des messages insignifiants dérangeaient sur le mobile.

Mécanique : Route les messages strictement selon la gravité : INFO -> toast UI, WARNING -> journal du tableau de bord, CRITICAL -> push Telegram immédiat.

Bénéfice : *Alarme ciblée sans fatigue de notification.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

55. Pipeline multimodal de vision et de capture d'écran (backend/vision_pipeline.py)

Problème : Les problèmes visuels ou tableaux dans les images devaient être décrits laborieusement.

Mécanique : Capture d'écran directe dans le navigateur (bouton caméra dans le chat) ainsi qu'analyse OCR et structurelle automatique pour les images téléversées (.png, .jpg, .webp).

Bénéfice : *Les agents voient exactement ce que tu vois à l'écran.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

56. Moteur vocal Apollo — Edge-TTS et dictée Whisper (apollo_voice_engine.py)

Problème : Saisir de longs textes sur smartphone ou desktop coûte du temps ; on préfère écouter les rapports en déplacement.

Mécanique : Pipeline vocal bidirectionnel : dictée au microphone (speech-to-text) directement dans le champ de saisie du chat et sortie vocale naturelle (text-to-speech) pour les briefings. Intégration vocale Telegram complète (POST /api/telegram/voice-relay et POST /api/voice/synthesize) pour des mémos audio mobiles via edge-tts (de-DE-ConradNeural).

Bénéfice : *Transforme Archon OS en assistant exécutif à commande vocale.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active

d'actions LLM au sein du module).

57. Radar d'actualités Oracle et agent d'exécution SEO

Problème : Les évolutions du marché et les faiblesses SEO de ses propres pages web n'étaient contrôlées que sporadiquement.

Mécanique : Le radar Oracle scrute en continu les sources spécialisées pour leur pertinence de projet ; l'agent SEO contrôle les pages sur les méta-tags, la structure et la densité de mots-clés.

Bénéfice : *Optimisation continue du marché et de la visibilité en arrière-plan.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

7. Mémoire, hygiène des bases de données et Obsidian 2nd Brain (pilier 4)

58. Couche mémoire SQLite WAL centrale (5 tables standard)

Problème : Des accès simultanés en lecture et écriture de plusieurs agents entraînaient des conflits avec de simples fichiers.

Mécanique : Base SQLite transactionnelle (data/agentic_memory.db) en mode Write-Ahead-Logging (WAL) avec les 5 tables centrales tasks, conversations, learnings, kv_store et user_profile.

Bénéfice : *Accès parallèles sans blocage avec sécurité maximale des données.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

59. Sidecar Kanban à l'épreuve des processus avec échange atomique de fichiers (backend/kanban_sidecar.py)

Problème : Des métadonnées comme les limites WIP ou les tags pouvaient se corrompre lors d'écritures parallèles.

Mécanique : Protège config/kanban_meta.json via un mutex à l'échelle du processus et des opérations atomiques write-replace.

Bénéfice : *Prévient toute perte de données lors des actualisations haute fréquence du panneau.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

60. Pont RAG Docker vers AnythingLLM (5 068 documents indexés sur le port 3001)

Problème : Les connaissances présentes dans le stockage vectoriel Docker n'étaient souvent pas mobilisées automatiquement lors de requêtes de chat normales.

Mécanique : backend/rag_bridge.py relie le noyau des agents directement au workspace AnythingLLM ARCHON et fournit les passages pertinents comme contexte ancré.

Bénéfice : *Des réponses étayées sur la base de tes vrais documents d'entreprise plutôt que d'estimations générales.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

61. Répartition des dossiers de projet à trois niveaux dans l'Obsidian-Vault

Problème : Quand les agents créent des fichiers, le désordre s'installe vite dans le coffre sans règles fixes.

Mécanique : Crée automatiquement pour chaque projet la structure à 3 niveaux : 01_Zwischenberichte, 02_Medien et 03_Endprodukte.

Bénéfice : *Séparation nette entre brouillons bruts et résultats finaux réceptionnés.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de

bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

62. Indexeur Obsidian-Vault et Map of Content automatique (backend/vault_hygiene.py)

Problème : Les nouveaux rapports de projet restaient souvent non liés dans le graphe Obsidian (« notes orphelines »).

Mécanique : Scanne le coffre, repère les notes orphelines et actualise de façon entièrement automatique le _MOC_Projekt_Index.md central avec des wikiliens.

Bénéfice : *Maintient le graphe de connaissances dans Obsidian toujours pleinement interconnecté.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

63. Moissonnage autonome de faits et déduplication à 85 % de similarité (backend/post_task_pipeline.py)

Problème : Des constats importants issus de tâches terminées se perdaient ou étaient enregistrés plusieurs fois en double.

Mécanique : Extrait automatiquement après clôture de tâche de nouveaux faits pour learnings et user_profile et évite les doublons dès 85 % de similarité textuelle.

Bénéfice : *Le système devient plus intelligent à chaque tâche accomplie sans encombrer la mémoire.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

64. Sauvegarde automatique de BD en ligne avec rotation à 7 jours (backend/db_backup_manager.py)

Problème : Sans sauvegardes régulières des bases, un défaut matériel menace de faire perdre toutes les tâches et tous les profils.

Mécanique : Exécute en cours d'exploitation PRAGMA wal_checkpoint(TRUNCATE) et enregistre des archives .db.gz compressées dans .hermes/backups/db/.

Bénéfice : *Sauvegarde complète des données sans une seconde d'arrêt.*

Prix et revers (arbitrage) : Les sauvegardes complètes quotidiennes occupent ~50–500 Mo d'espace disque selon la durée de conservation.

Quand désactiver ? et seuils : Standard : 7 jours de conservation (db_backup_keep_days).

65. Vacuum SQLite autonome et optimiseur de fragmentation (backend/db_optimizer.py)

Problème : Les enregistrements supprimés et les mises à jour fréquentes gonflent inutilement les fichiers SQLite au fil des mois.

Mécanique : Exécute via POST /api/system/optimize-db PRAGMA optimize et VACUUM sur toutes les bases et calcule l'espace libéré.

Bénéfice : *Maintient durablement les temps de requête dans la plage des millisecondes.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

66. Instantanés atomiques pré-écriture et rollback en 1 clic (backend/snapshot_manager.py)

Problème : Quand un agent écrase fautivement un fichier, la restauration de l'état antérieur était laborieuse.

Mécanique : Sauvegarde automatiquement l'original dans .hermes/snapshots/ avant chaque accès en écriture et le restaure aussitôt par clic ou API. Intégré en outre comme instantanés pré-update de toutes les bases SQLite avant les pulls GitHub et les mises à jour de conteneurs dans data/snapshots/.

Bénéfice : *Filet de sécurité complet pour toutes les modifications de fichiers et de code.*

Prix et revers (arbitrage) : Chaque instantané pré-écriture coûte ~5–15 ms d'I/O disque avant l'écrasement d'un fichier.

Quand désactiver ? et seuils : Tampon circulaire limité par standard à 50 instantanés.

67. Aperçu Live-Diff compatible Git (backend/diff_engine.py)

Problème : Avant une validation, on veut voir exactement quelles lignes changent au lieu de relire tout le fichier.

Mécanique : Génère une comparaison de lignes en couleur (rouge = supprimé, vert = nouveau) entre l'existant et le brouillon de l'agent.

Bénéfice : *Contrôle en quelques secondes des modifications de code et de documents dans le portail HITL.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

8. Sécurité, gouvernance Zero-Trust et diagnostic système (pilier 6)

68. Journal d'audit SHA-256 chaîné cryptographiquement (backend/audit_log.py)

Problème : De simples journaux texte peuvent être modifiés ou supprimés après coup.

Mécanique : Chaque événement est chaîné avec le hachage SHA-256 de l'entrée précédente (principe blockchain) et est vérifiable contre toute manipulation via GET /api/audit/verify.

Bénéfice : *Preuve révisable de toutes les actions système et de validation.*

Prix et revers (arbitrage) : Surcoût d'écriture minimal (~2 ms par entrée) ; la chaîne cryptographique croît au fil des mois.

Quand désactiver ? et seuils : Laisser durablement activé pour une gouvernance révisable.

69. Garde de sécurité locale — 25 signatures d'injection de prompts et de jailbreak

Problème : Des e-mails ou pages web malveillants peuvent contenir des instructions cachées (« Ignore previous instructions... »).

Mécanique : Scanne chaque entrée de texte externe selon 25 modèles d'attaque avant qu'elle n'atteigne le contexte d'un agent.

Bénéfice : *Repousse les injections indirectes de prompts à la frontière du système.*

Prix et revers (arbitrage) : Des heuristiques très strictes peuvent déclencher de fausses alertes sur des textes de sécurité informatique ou des prompts de pen-testing.

Quand désactiver ? et seuils : N'assouplir que ponctuellement pour des tests Red-Teaming autorisés.

70. Masquage de secrets et garde de sortie anti-red-teaming (backend/secret_masker.py)

Problème : Des clés API ou mots de passe pourraient apparaître par inadvertance dans des journaux, rapports ou sorties de chat.

Mécanique : Filtre les textes sortants en temps réel sur les modèles de clés (sk-..., Bearer, mots de passe) et les remplace par [GESCHÜTZTER_KEY].

Bénéfice : *Empêche la divulgation accidentelle d'identifiants confidentiels.*

Prix et revers (arbitrage) : Un masquage regex agressif peut dans de rares cas noircir des chaînes Base64 ou de hachage inoffensives dans les sorties de code.

Quand désactiver ? et seuils : Laisser durablement actif en production pour protéger les clés API.

71. Proxy de socket Docker sécurisé (port 2375 interne)

Problème : L'accès direct à /var/run/docker.sock donne aux conteneurs des droits root illimités sur le système hôte.

Mécanique : Intercale un conteneur proxy restrictif qui ne laisse passer que les ordres de statut et de redémarrage explicitement autorisés.

Bénéfice : *Protège le système d'exploitation hôte selon le principe Zero-Trust.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active

d'actions LLM au sein du module).

72. Comparateur de modèles A/B et scoreur empirique de performance (backend/model_benchmarker.py)

Problème : De nouveaux modèles de langage paraissent chaque semaine ; leur aptitude à ses propres tâches d'entreprise reste souvent floue.

Mécanique : Teste deux modèles en parallèle avec des tâches identiques et compare temps de réponse, coûts en jetons et qualité des résultats.

Bénéfice : *Base de données objective pour le choix de modèles dans le cadre des agents.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

73. Score de santé 360° à l'échelle du système et moteur de diagnostic (backend/system_diagnostics.py)

Problème : Des métriques isolées ne montrent souvent pas d'un coup d'œil la santé réelle de l'ensemble du système.

Mécanique : Calcule à partir de l'état de flotte Docker (35 %), de la file de concurrence (20 %), de l'intégrité d'audit (20 %), de l'absence de deadlock (15 %) et de l'accessibilité RAG (10 %) un score global de 0–100 % avec recommandations d'action.

Bénéfice : *Diagnostic global immédiat du système d'exploitation via GET /api/system/health-score.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

9. Auto-évolution, marchés et auto-documentation (pilier 7)

74. Synthèse autonome de compétences avec rétroaction aux dossiers personnels (backend/skill_synthesizer.py)

Problème : Les voies de résolution multi-niveaux réussies devaient jusqu'ici être programmées manuellement comme nouvelle compétence.

Mécanique : Détecte les chaînes d'outils réussies récurrentes, génère automatiquement un projet SKILL.md, l'inscrit dans le dossier personnel de l'agent et le soumet pour validation.

Bénéfice : *Le système étend ses capacités en exploitation de façon autonome.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

75. Marché dynamique de talents et de compétences (skill_markt.js et AgentSkillsHub)

Problème : Écrire de zéro de nouveaux rôles ou workflows pour de nouveaux domaines d'affaires prend du temps.

Mécanique : Catalogue interrogeable pour l'import et l'export en 1 clic de profils d'agents complets ainsi que de compétences externes (y compris intégration agentskillshub.dev).

Bénéfice : *Met la flotte d'agents à l'échelle en quelques secondes pour de nouveaux projets.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

76. Synchronisation complète frontend-backend (238 fetches / 581 chemins / 29 modales)

Problème : Dans les grandes applications web, des boutons orphelins ou des modales non connectées conduisent souvent à des erreurs silencieuses.

Mécanique : Un pipeline de contrôle automatisé garantit que les 238 appels API frontend et 29 modales sont câblés exactement avec les 368 routes backend.

Bénéfice : *Garantit 0 lien mort et 100 % de contrôles utilisables dans toutes les vues.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

77. Générateur autonome de dossier maître et de manuel (backend/system_dossier_generator.py)

Problème : Les documentations d'architecture deviennent généralement obsolètes quelques jours après leur création.

Mécanique : Génère sur pression de bouton (POST /api/system/generate-dossier) le dossier d'architecture système actuel simultanément dans le dossier de projet et dans l'Obsidian-Vault (00_DOKUMENTATION/).

Bénéfice : *Maintient durablement la documentation système à jour sans effort d'entretien manuel.*

Prix et revers (arbitrage) : Intégré comme vue centrale permanente ou infrastructure système dans le tableau de bord (besoin de base RAM/CPU négligeable ; les coûts en jetons ne surviennent que lors de l'exécution active d'actions LLM au sein du module).

10. Matrice de référence : plugins Evey/Hermes pilotables

N° et ID de plugin	Catégorie spécialisée	Mécanique centrale et fonction dans le système d'exploitation
78. evey-cost-guard	Sécurité et gardiens	Surveillance des coûts en temps réel et coupure budgétaire dure par jour et par tâche.
79. evey-session-guard	Sécurité et gardiens	Surveille les durées de session, les plafonds de jetons et prévient les sessions zombies.
80. evey-email-guard	Sécurité et gardiens	Scanne les e-mails entrants avant triage contre hameçonnage et injections de prompts.
81. evey-watchdog	Sécurité et gardiens	Contrôle toutes les 30 s les processus actifs et redémarre automatiquement les workers bloqués.
82. evey-cache	Performance et cache	Cache de réponses de 24h pour répondre immédiatement et gratuitement aux requêtes répétées.
83. disk-cleanup	Performance et maintenance	Nettoie les artefacts temporaires, les journaux périmés et libère de l'espace disque.
84. evey-telemetry	Supervision et analytique	Enregistre les flux de jetons, latences et appels de modèles pour Mission Control et Guardian.
85. evey-status	Supervision et analytique	Ping de santé continu sur les 11 conteneurs Docker et passerelles.
86. evey-delegation-score	Supervision et analytique	Évalue les taux de succès et de qualité des transferts de tâches entre agents.
87. evey-autonomy	Intelligence des agents	Pilote le degré d'autonomie dans les décisions de fond multi-niveaux.
88. evey-council	Intelligence des agents	Formation de majorités et de consensus entre plusieurs spécialistes (couplage Mastermind).
89. evey-reflect	Intelligence des agents	Boucle d'auto-réflexion pour correction autonome des

		erreurs avant remise du résultat.
90. evey-learner	Intelligence des agents	Extrait des constats durables des tâches terminées pour la table learnings.
91. evey-identity	Intelligence des agents	Assure le strict respect de SOUL.md, de la tonalité et des garde-fous de rôle.
92. evey-habits	Intelligence des agents	Gère les routines récurrentes comme le briefing standup quotidien de 08h00.
93. evey-proactive	Intelligence des agents	Permet aux agents de proposer eux-mêmes des actions quand des goulots sont détectés.
94. evey-goals	Planification et tâches	Relie les tâches opérationnelles isolées du Kanban aux objectifs supérieurs de l'entreprise.
95. evey-scheduler	Planification et tâches	Planification des ordres de fond pilotée par le temps et les priorités.
96. evey-bridge	Ponts et interfaces	Plaque tournante interne des données entre le monolithe FastAPI, le démon Hermes et les sidecars.
97. evey-mqtt	Ponts et interfaces	Protocole de messages léger pour signaux de statut IoT et temps réel.
98. evey-moltbook	Ponts et interfaces	Synchronise les classeurs structurés et les notes de synthèse.
99. evey-telegram-ux	Ponts et interfaces	Pilote le routage par noms d'appel (/robert, /anwalt) et les boutons de validation interactifs dans Telegram.
100. evey-github	Ponts et interfaces	Connexion pour opérations de dépôt, analyses de commits et suivi des issues.
101. evey-research	Outils et recherche	Pilote de recherche profonde pour scraping web et extraction de sources dans le Research Lab.
102. evey-news	Outils et recherche	Scanner de fond pour flux sectoriels et actualités de marché (radar Oracle News).
103. evey-digest	Outils et recherche	Compresse de volumineuses collections documentaires en résumés compacts pour la direction.
104. evey-sandbox	Outils et exécution	Environnement d'exécution isolé pour le code Python et d'analyse généré dans le Showroom.
105. evey-rag	Mémoire et connaissances	Pilote de récupération vectorielle pour connecter AnythingLLM et les archives de connaissances locales.
106. evey-memory-adaptive	Mémoire et connaissances	Pondération dynamique de la mémoire à court et long terme selon la pression du contexte.
107. evey-memory-consolidate	Mémoire et connaissances	Traitement nocturne de consolidation et déduplication des entrées de mémoire liées.
108. evey-wallet	Finances et budgets	Gère les crédits API, contingents et limites de transactions liées aux coûts.

109. evey-validate	Qualité et exécution	Contrôle les sorties sur la conformité au schéma, les marqueurs obligatoires (SYNTHESE-BEREIT) et les prescriptions de format.
110. evey-verification	Qualité et exécution	Vérification des faits et contre-contrôle des affirmations critiques avant validation finale.
111. evey-delegate-model	Qualité et exécution	Choisit automatiquement pour les sous-tâches le modèle cible optimal en coût et performance.

11. Architecture d'entreprise et de résilience

78. Débogage Replay et Time-Travel (backend/time_travel_debugger.py)

Problème : Avec des workflows d'agents complexes à nombreux appels d'outils, le diagnostic d'erreurs a posteriori dans le journal texte est confus et la ramification des fils d'exécution impossible.

Mécanique : Enregistre chaque étape partielle avec entrées/sorties, instantanés et consommation de jetons dans « execution_traces ». Permet le replay interactif (GET /api/traces/{task_id}), le fork d'état à partir de l'étape N (POST /api/traces/{task_id}/fork) et le rollback en 1 clic des fichiers modifiés.

Bénéfice : Analyse rapide des causes et expériences de type « et si » sans risque avec des prompts ou modèles alternatifs, sans redémarrer tout le pipeline.

Prix et revers (arbitrage) : Augmente l'I/O disque d'environ 5–15 ms par édition de fichier et occupe selon la limite d'instantanés ~50–250 Mo d'espace supplémentaire sur le disque.

Quand désactiver ? et seuils : Désactiver sur les systèmes à espace disque extrêmement restreint ou en cas de générations massives de fichiers (p. ex. artefacts de build). | Seuils : Max instantanés : < 10 = historique court | > 200 = consommation disque accrue (std : 50). Max étapes de trace : std 100.

79. Matrice d'éval multi-modèles et comparateur A/B (backend/model_benchmarker.py)

Problème : Les affectations de modèles aux agents reposent souvent sur des hypothèses subjectives faute de valeurs de comparaison empiriques.

Mécanique : Envoie le même prompt simultanément à deux LLM configurés (POST /api/router/benchmark), mesure latence, consommation de jetons et longueur de réponse et stocke durablement l'historique dans « model_benchmarks » (GET /api/router/benchmark/history).

Bénéfice : Choix de modèles objectif et fondé sur les données par rôle d'agent (p. ex. DeepSeek V3 pour rapidité/coûts vs Claude 3.7 pour architectures complexes).

Prix et revers (arbitrage) : N modèles candidats coûtent N fois les jetons API par cycle de test.

Quand désactiver ? et seuils : Quand désactiver ? Ne lancer que ciblément pour des comparaisons de modèles, pas en continu.

80. Portail de consensus à trois agents et tribunal arbitral (backend/consensus_gate.py)

Problème : Les interventions critiques (refactorings de code, migrations) comportent en décision isolée un risque d'hallucinations ou de failles de sécurité.

Mécanique : Convoque un comité de trois membres (architecte système, responsable sécurité, lead QA) (POST /api/consensus/council-vote). Chaque réviseur vote indépendamment ; la validation exige une majorité aux 2/3. Toutes les décisions sont archivées dans « consensus_decisions » (GET /api/consensus/history).

Bénéfice : Minimisation fiable des risques et élimination des hallucinations à point de défaillance unique lors de modifications système sensibles.

Prix et revers (arbitrage) : Triple (3x) la consommation de jetons et augmente le temps de réponse de +6 à +20 s. En cas d'exigence d'unanimité, risque de blocage décisionnel (deadlock).

Quand désactiver ? et seuils : Désactiver (ou n'activer qu'à partir de la priorité 4 = critique) quand la vitesse et les faibles coûts API priment. | Seuils : à partir de priorité : prio 2 = presque chaque tâche coûte 3x jetons | prio 4 = seules tâches critiques (std : prio 3). Majorité : majorité aux 2/3 recommandée.

81. Auto-guérison de tâches autonome avec rollback Git (backend/task_self_healer.py)

Problème : Les erreurs de syntaxe ou imports de code fautifs bloquent aussitôt les tâches et exigent une intervention manuelle de l'utilisateur.

Mécanique : Isole la trace de pile, effectue en cas d'erreurs d'exécution un rollback Git des fichiers modifiés et transmet à l'agent un prompt de correction structuré (POST /api/healing/attempt). Jusqu'à 3 tentatives de réparation autonomes ; récupération des journaux via GET /api/healing/logs.

Bénéfice : Résilience maximale pour les exécutions de code et de scripts avec auto-réparation avant l'escalade vers le panneau HITL.

Prix et revers (arbitrage) : Chaque tentative d'auto-guérison coûte 1 appel LLM supplémentaire (+3 à +12 s de latence et coûts en jetons). Un rollback Git automatique écarte aussi en cas d'échec des succès partiels utiles.

Quand désactiver ? et seuils : Désactiver le rollback Git quand des brouillons de code inachevés doivent rester dans le répertoire de travail pour poursuite manuelle. | Seuils : Max tentatives de réparation : 1 = abandonne aussitôt | > 5 = cascade de réparation coûteuse (std : 3).

82. Maille événementielle et observateur de répertoire d'entrée (backend/event_mesh.py)

Problème : Les tâches devaient jusqu'ici être lancées manuellement via le tableau de bord ou interrogées via polling IMAP planifié.

Mécanique : Bus central d'événements entrants pour webhooks externes (POST /api/events/inbound/{channel}) et observateur automatique de répertoire (POST /api/events/scan-directory), qui convertit les nouveaux fichiers dans « data/inbox/ » directement en tâches de triage Kanban.

Bénéfice : Enregistrement entièrement automatique d'événements et de pièces depuis des systèmes externes (GitHub, n8n, Stripe, scanners de fichiers locaux) sans interaction utilisateur.

Prix et revers (arbitrage) : Des intervalles de scrutation très courts (< 3 s) génèrent des polls continus du système de fichiers ; le standup matinal quotidien consomme ~1 500–3 000 jetons par exécution.

Quand désactiver ? et seuils : Désactiver si le dossier d'entrée local n'est pas activement utilisé ou si aucun rapport de briefing matinal n'est souhaité. | Seuils : intervalle de scrutation : 1–2s = polls disque inutiles | > 30s = détection de fichiers retardée (std : 5s). Heure du standup : std 08h00.

12. Architecture d'autonomie, de bac à sable et de gouvernance

83. Bacs à sable de code éphémères (backend/ephemeral_sandbox.py)

Problème : L'exécution de code d'agents comporte un risque de plantages système, de boucles infinies ou de modifications de fichiers involontaires sur l'hôte.

Mécanique : Exécute le code dans des micro-conteneurs Docker isolés à courte vie avec limitation stricte des ressources (512 Mo RAM, 1 CPU, timeout 30s, pas de réseau) (POST /api/sandbox/execute). Le bac à sable est supprimé sans laisser de traces après achèvement.

Bénéfice : Sécurité absolue et stabilité système pour l'exécution et l'essai automatisés de code.

Prix et revers (arbitrage) : Le démarrage à froid du conteneur coûte ~300–800 ms de latence supplémentaire par appel d'outil. Si le réseau est désactivé, pip install ou les appels API externes échouent aussitôt dans le script.

Quand désactiver ? et seuils : Désactiver (ou activer le réseau) quand des scripts locaux de confiance ont besoin d'un accès hôte direct ou d'installations rapides de paquets. | Seuils : limite RAM : < 256 Mo = OOM-kill avec Pandas/NumPy | > 2048 Mo = risque RAM hôte (std : 512 Mo). Timeout : < 10s = interrompt les builds (std : 30s).

84. Consolidation nocturne de la mémoire (backend/memory_consolidator.py)

Problème : Les traces, battements de cœur et données de dialogue périmées qui croissent en continu ralentissent les requêtes de base en exploitation permanente.

Mécanique : Élague pendant les phases calmes (standard 03h00) les traces périmées et les journaux non critiques selon des délais de rétention configurables (settings.json). Entièrement pilotable via les réglages Hermes Core (heure, jours de rétention, profondeur d'élagage, bouton d'exécution manuelle).

Bénéfice : Des bases durablement allégées, des latences réduites et une hygiène système automatique sans effort de

maintenance manuelle.

Prix et revers (arbitrage) : Pendant l'exécution de consolidation (~10–30 s), brève charge CPU/disque ; un seuil d'élagage trop bas (< 0.88) peut fusionner des faits liés mais autonomes.

Quand désactiver ? et seuils : Désactiver si des traces brutes complètes doivent rester durablement non filtrées pour un audit forensique. | Seuils : conservation : < 7 jours = perte de traces d'audit importantes | > 60 jours = grosse BD SQLite (std : 14 jours). Seuil d'élagage : 0.95 = conservateur | 0.88 = agressif (std : 0.92).

85. Agent web visuel avec audit par captures (backend/visual_web_agent.py)

Problème : Les tâches sur des portails web complexes sans API (SPA, formulaires, tableaux de bord interactifs) ne peuvent être traitées avec des scrapers texte.

Mécanique : Pilote un vrai navigateur headless Chromium (POST /api/browser/browse-and-capture), enregistre des captures pour chaque étape de clic et de saisie dans .hermes/visual_traces/{task_id}/ et extrait des données structurées.

Bénéfice : Traçabilité visuelle sans faille et interaction automatisée avec des applications web modernes sans accès API.

Prix et revers (arbitrage) : Chromium headless requiert par cycle d'audit actif ~300 Mo RAM et ~3–5 s de temps de rendu.

Quand désactiver ? et seuils : Quand désactiver ? Pour une exploitation purement backend sans contrôles UI.

86. War-Room virtuelle et test de résistance pré-mortem (backend/war_room_simulator.py)

Problème : Des risques imprévus de sécurité, d'acceptation ou de conformité sont souvent remarqués après le déploiement lors des lancements de produits.

Mécanique : Simule trois parties adverses (gros client critique, pentester offensif, auditeur de conformité strict) avant validation (POST /api/simulation/war-room). Établit une matrice de risques agrégée (LOW à CRITICAL) avec prescriptions contraignantes.

Bénéfice : Mise au jour proactive des faiblesses et prévention de coûteuses mauvaises décisions avant la mise en production.

Prix et revers (arbitrage) : Coûte ~1 500–3 000 jetons supplémentaires et +4 à +10 s d'attente avant la tâche proprement dite. Avec un seuil de blocage MEDIUM, des fausses alertes plus fréquentes.

Quand désactiver ? et seuils : Désactiver l'auto-pré-mortem dans le quotidien de développement normal et ne l'utiliser que pour des interventions critiques en production (HIGH/CRITICAL). | Seuils : seuil de blocage : MEDIUM = bloque souvent par précaution | CRITICAL = n'arrête que les risques catastrophiques (std : HIGH).

87. Contrats SLA et de transfert entre agents (backend/sla_contract_enforcer.py)

Problème : Des délégations lâches dans les hiérarchies multi-agents conduisent souvent à des contributions incomplètes qui sont escaladées vers l'opérateur humain.

Mécanique : Conclut lors de délégations internes des contrats numériques avec champs obligatoires, longueur minimale et critères d'acceptation (POST /api/contracts/create). Valide les livraisons de façon autonome (POST /api/contracts/validate) et renvoie automatiquement les résultats insuffisants pour retouche.

Bénéfice : Respect strict des normes de qualité dans l'équipe d'agents et prévention d'étapes intermédiaires inachevées dans le panneau HITL.

Prix et revers (arbitrage) : Si la longueur minimale est fixée trop haut (> 300 caractères), des réponses courtes précises sont rejetées et déclenchent de coûteuses boucles de retouche.

Quand désactiver ? et seuils : Désactiver l'auto-rejet quand les sous-agents peuvent aussi renvoyer des réponses de statut ou oui/non succinctes. | Seuils : longueur min. : < 30 = laisse passer les espaces réservés | > 300 = rejette les réponses factuelles succinctes (std : 100 caractères). Délai max : std 60 min.

88. Pilotage central de l'autonomie et des seuils (backend/autonomy_settings.py)

Problème : Des seuils fixes dans le code empêchent une adaptation rapide des règles de sécurité, de bac à sable et de résilience en cours d'exploitation.

Mécanique : Regroupe les 10 groupes d'autonomie (concurrence, tueur de boucles, auto-guérison, bac à sable Docker, instantanés Time-Travel, portail de consensus, War-Room, contrats SLA, entrée/standup et hygiène de

flotte) dans settings.json et les synchronise via GET/POST /api/settings/autonomy-modules avec le System Control Center.

Bénéfice : Chaque module de protection et d'autonomie peut être activé ou désactivé au clic pendant l'exécution et ajusté exactement au profil de risque et de ressources souhaité.

Prix et revers (arbitrage) : Un conteneur défectueux peut entrer en boucle de crash via l'auto-restart ; l'audit visuel du navigateur lance Chromium (+300 Mo RAM, +4 s de durée).

Quand désactiver ? et seuils : Désactiver l'auto-restart pendant la maintenance manuelle des conteneurs ; désactiver l'audit navigateur pour une exploitation purement backend. | Seuils : jours de sauvegarde BD : < 3 jours = historique juste | > 30 jours = ~500 Mo+ de stockage de sauvegarde (std : 7 jours).

13. Cœur de performance, mise à l'échelle du débit et efficacité des jetons

89. Mise en cache sémantique de prompts et réponses (backend/semantic_cache.py)

Problème : Des requêtes d'agents récurrentes ou légèrement reformulées causent des coûts API LLM inutiles et des secondes d'attente.

Mécanique : Calcule pour chaque prompt un hachage SHA-256 ainsi qu'un vecteur mot/3-grammes normalisé L2 dans SQLite et fournit dès une similarité cosinus >= 0.95 aussitôt le résultat enregistré.

Bénéfice : Temps de réponse sous 10 millisecondes et 100 % d'économie de jetons pour des requêtes sémantiquement identiques.

Prix et revers (arbitrage) : Si le seuil cosinus est trop bas (< 0.92), le cache néglige de fines différences de sens (p. ex. « supprime la BD de test » vs « supprime la BD de prod ») et fournit des réponses fausses ou périmées.

Quand désactiver ? et seuils : Désactiver pour les tâches à données temps réel (cours boursiers, journaux en direct, état système) ou quand chaque réponse doit être générée fraîchement. | Seuils : seuil cosinus : < 0.92 = fausses réponses du cache ! | > 0.98 = guère de résultats (std : 0.95). TTL : std 24h. Entrées max : std 500.

90. Sous-tâches parallèles Map-Reduce / fan-out et fan-in (backend/map_reduce_executor.py)

Problème : Le traitement séquentiel de nombreux documents isolés ou sous-questions conduit à une croissance linéaire du temps et à de longues attentes.

Mécanique : Répartit automatiquement les ordres de listes sur jusqu'à N workers Asyncio parallèles (pilotés par sémaphore) et réunit les résultats partiels dans l'étape Reduce.

Bénéfice : Multiplie le débit pour les analyses batch de documents, recherches et code sans surcharger la boucle d'événements.

Prix et revers (arbitrage) : Génère une charge API simultanée N fois supérieure (risque de limites HTTP-429 chez le fournisseur) et ~20–35 % de surcoût en jetons via la synthèse Reduce ultérieure.

Quand désactiver ? et seuils : Désactiver en cas de limites API strictes ou quand les sous-tâches dépendent étroitement les unes des autres et doivent s'enchaîner séquentiellement. | Seuils : Max workers : > 8 = risque de limite de débit (std : 4). Timeout de bloc : std 30s. À partir d'éléments : std 3.

91. Routage rapide spéculatif et couverture de latence (backend/speculative_hedger.py)

Problème : Des requêtes cloud isolées lentes (pics de latence de queue) bloquent toute l'exécution de l'agent.

Mécanique : Lance après écoulement de hedge_delay_seconds (p. ex. 2.5s) automatiquement un appel de remplacement parallèle sur un modèle de repli rapide et retient le résultat arrivé en premier.

Bénéfice : Élimine les blocages dus aux points de terminaison API lents et stabilise les temps de réponse au 99e percentile.

Prix et revers (arbitrage) : Si le modèle principal répond quand même peu après le lancement de la couverture, les deux requêtes sont facturées (jusqu'à 2x coûts en jetons pour les prompts lents).

Quand désactiver ? et seuils : Désactiver en cas de budget API restreint ou quand des modèles de raisonnement lents sont utilisés, qui requièrent régulièrement > 10 s de réflexion. | Seuils : délai de couverture : < 1.5s = presque toujours coûts API doubles ! | > 8s = guère de protection de latence (std : 2.5s).

92. Compaction dynamique de contexte et résumé hiérarchique (backend/context_compactor.py)

Problème : De longues sessions d'agents gonflent la fenêtre de contexte, ralentissent le pré-remplissage du prompt et augmentent drastiquement les coûts par tour.

Mécanique : Extrait à partir d'un seuil de jetons réglable tous les faits durs (objectifs, chemins de fichiers, décisions, erreurs) dans un bloc compact [HIERARCHICAL_MEMORY_STATE] et ne conserve en texte intégral que les tours les plus récents.

Bénéfice : Économise 65–80 % de jetons de contexte pour les longues sessions sans perte de données de projet importantes.

Prix et revers (arbitrage) : La passe de compression coûte ponctuellement ~1 000 jetons ; des lignes de code exactes ou petites valeurs de détail de messages très anciens peuvent se perdre dans le résumé.

Quand désactiver ? et seuils : Désactiver quand l'agent doit recourir mot pour mot à des messages très anciens de la même session. | Seuils : seuil de jetons : < 3000 = oublie trop tôt les détails | > 24000 = coûts de prompt élevés (std : 6000). Derniers tours conservés : std 4.

93. Pool de connexions SQLite WAL et écriture batch asynchrone (backend/async_write_batcher.py)

Problème : Des centaines d'INSERT SQLite isolés lors d'exécutions d'agents parallèles génèrent des pics d'I/O disque et des conflits « database is locked ».

Mécanique : Collecte les événements de télémétrie et d'audit dans un tampon circulaire RAM thread-safe et les écrit de façon groupée via executemany() en mode WAL.

Bénéfice : Réduit les accès en écriture disque d'environ 80 % et maintient la boucle d'événements principale libre de blocages.

Prix et revers (arbitrage) : En cas de coupure de courant dure ou de kill -9 avant le prochain intervalle de flush, les entrées tamponnées en RAM des dernières 1–5 secondes sont perdues.

Quand désactiver ? et seuils : Désactiver quand chaque entrée de journal doit reposer aussitôt et sans délai de secondes en synchrone sur le disque. | Seuils : taille de batch : < 5 = nombreuses écritures disque | > 100 = plus de perte en cas de crash (std : 25). Intervalle de flush : std 2.0s.

14. Architecture moderne de mémoire et RAG

94. Recherche hybride avec BM25 FTS5 et fusion de rangs réciproques (backend/hybrid_search_engine.py)

Problème : La pure recherche vectorielle trouve des textes sémantiquement similaires, mais échoue souvent sur des codes d'erreur exacts, des noms de fonctions ou des numéros de série.

Mécanique : Combine le classement plein texte SQLite FTS5 (BM25) et la similarité vectorielle cosinus via la fusion de rangs réciproques (RRF).

Bénéfice : Concilie résultats exacts par mots-clés et compréhension sémantique en une seule liste de résultats hautement précise.

Prix et revers (arbitrage) : La double interrogation d'index augmente la latence de recherche de ~15–40 ms ; un Top-K trop élevé (> 10) remplit le prompt de l'agent de milliers de jetons supplémentaires.

Quand désactiver ? et seuils : Ne presque jamais désactiver — tout au plus réduire Top-K à 3–5 quand les coûts en jetons par requête doivent être minimisés. | Seuils : constante RRF K : std 60. Poids BM25 : 0.0 = vecteur seul | 0.5 = équilibre | 1.0 = mot-clé seul (std : 0.50). Top-K : std 5.

95. Graphe de connaissances local / magasin de triplets (backend/knowledge_graph_store.py)

Problème : Des blocs de texte plats ne saisissent pas de dépendances structurées (qui administre quel système ou quel composant utilise quel service).

Mécanique : Extrait et stocke des triplets sujet-prédicat-objet avec score de confiance dans SQLite et permet des traversées de graphe multi-niveaux (multi-sauts).

Bénéfice : Les agents reconnaissent les interdépendances système et organisationnelles sur plusieurs niveaux.

Prix et revers (arbitrage) : L'extraction automatique de triplets coûte ~500 jetons supplémentaires à l'enregistrement. Au-delà de 2 sauts de traversée, le nombre de nœuds voisins explose exponentiellement.

Quand désactiver ? et seuils : Désactiver l'auto-extraction pour du chat purement court terme sans construction de connaissances à long terme ; limiter les sauts à 2 au maximum. | Seuils : Max sauts : 1 = voisins directs | 2 = optimal | >= 3 = explosion de nœuds ! (std : 2). Confiance min. : std 0.70.

96. RAG correctif agentique / CRAG (backend/corrective_rag.py)

Problème : Le RAG classique transmet au modèle de langage des résultats non pertinents sans vérification et provoque ainsi des réponses erronées.

Mécanique : Évalue chaque bloc documentaire récupéré contre un seuil de pertinence (0.45) et effectue en cas de résultats faibles automatiquement une réécriture de requête avec expansion de synonymes.

Bénéfice : Filtre le bruit de connaissances avant la génération de réponses et augmente la qualité des résultats pour les questions floues.

Prix et revers (arbitrage) : Chaque reformulation de requête coûte 1 appel LLM supplémentaire (+1,5 à +4 s de latence). Si le seuil de pertinence est trop élevé (> 0.70), CRAG tourne en rond inutilement.

Quand désactiver ? et seuils : Désactiver quand la base de connaissances est petite et proprement tenue ou quand des temps de réponse inférieurs à 2 secondes sont exigés. | Seuils : seuil de pertinence : < 0.30 = laisse passer des sources faibles | > 0.70 = boucles de recherche sans fin (std : 0.45). Réécritures max : std 2.

97. Mémoire centrale à 2 niveaux sur le modèle MemGPT/Letta (backend/core_memory_manager.py)

Problème : Les préférences utilisateur importantes et les objectifs de projet se perdent dès qu'ils sortent de la fenêtre de chat éphémère.

Mécanique : Ancre durablement dans le prompt système des blocs éditables de mémoire de travail épinglée (<CORE_WORKING_MEMORY>), séparés du grand stockage d'archives.

Bénéfice : Garantit une mémoire centrale permanente, entretenue par l'agent lui-même, sur un nombre quelconque de sessions.

Prix et revers (arbitrage) : Chaque bloc épinglé occupe durablement des jetons de prompt à chaque appel LLM (10 blocs × 800 caractères ≈ 2 000 jetons de charge de base par message).

Quand désactiver ? et seuils : Désactiver ou réduire le nombre de blocs à 3–5 quand de petits modèles à fenêtre de contexte étroite ou des budgets de jetons restreints sont utilisés. | Seuils : Max blocs : > 15 = forte consommation de jetons de base (std : 10). Max caractères/bloc : < 300 = tronque les règles | > 1500 = engorge le prompt (std : 800).

98. Épinglage automatique de citations et de sources (backend/citation_pinner.py)

Problème : Sans références directes aux sources, l'utilisateur ne peut vérifier de quel document provient une affirmation factuelle de l'agent.

Mécanique : Attribue les phrases de la réponse aux documents sources récupérés par chevauchement de jetons, insère des preuves en ligne ([Source : ID]) et contrôle le quota minimal de preuves.

Bénéfice : Rend chaque affirmation retraçable de façon révisable et révèle aussitôt les allégations non étayées.

Prix et revers (arbitrage) : Augmente la consommation de jetons de sortie d'environ 10–15 % ; un taux de couverture minimal trop élevé (> 80 %) entraîne de fausses alertes pour les questions créatives ou de synthèse.

Quand désactiver ? et seuils : Désactiver pour le brainstorming libre, la génération de code ou le chat quotidien informel sans rapport documentaire. | Seuils : couverture min. : < 30 % = affirmations non étayées | > 85 % = fausses alertes sur résumés (std : 60 %).

15. Exécution fiable et déterminisme (Durable Execution)

99. Sorties structurées et garantie de schéma JSON (backend/structured_output_enforcer.py)

Problème : Les modèles de langage livrent parfois du JSON invalide (cadres Markdown, parenthèses manquantes ou types de données erronés), ce qui fait planter les pipelines de code en aval.

Mécanique : Répare automatiquement les erreurs de syntaxe dans la chaîne brute et impose tous les champs obligatoires et types de données du schéma cible transmis.

Bénéfice : Garantit des payloads JSON 100 % lisibles par machine et sûrs en types entre agents et interfaces backend.

Prix et revers (arbitrage) : Chaque boucle d'auto-réparation coûte +1 à +3 s de latence et des jetons supplémentaires. La validation stricte de schéma rejette les réponses même pour d'innocents champs additionnels.

Quand désactiver ? et seuils : Désactiver la validation stricte quand les points de terminaison API ou scripts s'accommodent aussi de champs JSON flexibles ou partiels. | Seuils : contrôle de schéma : strict vs tolérant (std : strict). Réparations max : 1–2 = répare 98 % des coquilles | > 3 = gaspille des jetons (std : 2).

100. Checkpointing d'exécution durable / pause et reprise (backend/durable_checkpoint_manager.py)

Problème : Quand un ordre de recherche ou de build en 10 étapes s'interrompt à l'étape 8 par un redémarrage, il fallait jusqu'ici répéter chèrement les 7 travaux préalables.

Mécanique : Sauvegarde après chaque étape achevée l'état complet des variables et de progression de façon transactionnelle dans SQLite.

Bénéfice : Les longues exécutions interrompues reprennent exactement au dernier checkpoint valide, sans consommer deux fois travail ni jetons.

Prix et revers (arbitrage) : Le checkpointing à chaque étape (N = 1) génère une charge d'écriture SQLite sensible. L'auto-resume au démarrage peut rallumer involontairement des tâches périmées de la veille.

Quand désactiver ? et seuils : Désactiver l'auto-resume quand les tâches interrompues doivent d'abord être contrôlées et validées manuellement après un redémarrage. | Seuils : tous les N étapes : 1 = absolument à l'épreuve des crashs, plus d'I/O disque | 3–5 = ménage le SSD (std : 1). Conservation : std 14 jours.

101. Clés d'idempotence et déduplication d'appels d'outils (backend/idempotency_guard.py)

Problème : En cas de retries réseau ou de boucles d'agents, le même e-mail, facture ou action webhook risque d'être déclenché plusieurs fois.

Mécanique : Attribue à chaque appel d'outil mutant une clé d'idempotence SHA-256 dans le registre SQLite et bloque les appels suivants identiques pendant la TTL.

Bénéfice : Prévient les effets extérieurs doubles de façon fiable sur le modèle des passerelles de paiement modernes (principe Stripe).

Prix et revers (arbitrage) : Si l'utilisateur veut exécuter intentionnellement exactement le même ordre une seconde fois pendant le délai de blocage (standard 24 h), le garde bloque.

Quand désactiver ? et seuils : Abaisser les heures de TTL à 1 h ou désactiver ponctuellement quand des actions identiques doivent être exécutées plusieurs fois de suite en phase de test. | Seuils : TTL (heures) : < 1h = courte protection anti-doublons | > 48h = bloque les répétitions voulues (std : 24h).

102. Essai à blanc / mode fantôme pour ordres d'agents (backend/dry_run_simulator.py)

Problème : Les ordres d'agents nouveaux ou complexes, on veut souvent d'abord les tester sans risque sur de vraies données de production avant exécution franche.

Mécanique : Intercepte en mode fantôme toutes les commandes d'écriture fichiers, bases de données et API et journalise à la place un plan d'aperçu de mutations détaillé.

Bénéfice : Permet des répétitions sans danger et des audits de risques avant la validation définitive.

Prix et revers (arbitrage) : TANT QU'IL EST ACTIF, AUCUN VRAI FICHIER N'EST ÉCRIT ET AUCUNE VRAIE API N'EST EXÉCUTÉE ! Si on oublie le commutateur, le système paraît « mort » vers l'extérieur.

Quand désactiver ? et seuils : Laisser impérativement DÉSACTIVÉ en exploitation productive normale ! Ne l'activer que ciblément pour des essais à blanc (dry-runs) et audits de sécurité. | Seuils : commutateur principal par défaut sur OFF (false). N'activer que pour des cycles de test isolés.

103. File de lettres mortes (DLQ) avec gestionnaire auto-replay (backend/dead_letter_queue.py)

Problème : Les webhooks échoués ou tâches de fond asynchrones se perdent sans trace sans bassin de recueil lors de pannes passagères de tiers.

Mécanique : Stocke chaque tâche échouée avec payload original et motif d'erreur dans la table DLQ et la rejoue avec backoff exponentiel.

Bénéfice : Aucun ordre ne se perd ; les tâches échouées peuvent être rejouées à tout moment isolément ou en groupe.

Prix et revers (arbitrage) : En cas d'ordres durablement fautifs (p. ex. clé API invalide), les tentatives de répétition consomment inutilement des ressources ; les replays différés peuvent déclencher des actions avec retard.

Quand désactiver ? et seuils : Désactiver l'auto-replay quand les ordres échoués doivent être exclusivement examinés manuellement et validés isolément. | Seuils : retries max : 1 = abandonne dès une courte panne | > 5 = accumule les tâches mortes (std : 3). Base de backoff : std 10s (10s -> 20s -> 40s).

16. Sécurité, protection des données et garde-fous d'entreprise

104. Nettoyeur PII bidirectionnel et réhydratation (backend/pii_scrubber.py)

Problème : Si des e-mails, numéros de téléphone ou IBAN sont transmis en clair à des LLM cloud externes, des violations du RGPD et des fuites de données involontaires menacent.

Mécanique : Remplace localement avant l'appel API les données personnelles sensibles par des espaces réservés ([EMAIL_1], [IBAN_1]) et les rééchange dans la réponse entrante contre les vraies valeurs.

Bénéfice : Les modèles cloud ne voient jamais de vraies données personnelles, tandis que l'utilisateur reçoit néanmoins sur place des documents et réponses complets.

Prix et revers (arbitrage) : Si le LLM modifie légèrement l'espace réservé dans le texte de réponse, la retraduction (réhydratation) échoue. De plus, le masquage regex peut parfois altérer des chaînes de code.

Quand désactiver ? et seuils : Désactiver quand seuls des modèles locaux (Ollama) sont utilisés ou que le LLM doit impérativement analyser les vrais domaines e-mail/noms. | Seuils : commutateurs granulaires pour e-mails, numéros de téléphone, IBAN et réhydratation automatique (tous actifs par standard).

105. Pare-feu sortant et liste blanche de domaines pour agents (backend/egress_firewall.py)

Problème : Via des injections de prompts cachées dans les pages web, les agents pourraient recevoir l'ordre d'envoyer des données internes par requête HTTP à des serveurs d'attaquants étrangers.

Mécanique : Contrôle chaque URL cible sortante contre la liste blanche de domaines configurée et bloque les hôtes non autorisés ainsi que les adresses IPv4 brutes.

Bénéfice : Coupe l'exfiltration de données au niveau réseau même en cas d'entrées manipulées.

Prix et revers (arbitrage) : Tant que allowlist_only est actif, chaque nouvelle page web, API ou documentation qui ne figure pas explicitement dans la liste de domaines est durement bloquée lors de recherches web.

Quand désactiver ? et seuils : Désactiver (ou compléter les domaines requis) quand l'agent de recherche doit fouiller librement le web ouvert. | Seuils : domaines autorisés : liste blanche séparée par virgules. Le blocage des IP brutes protège contre les exfiltrations IP directes.

106. RBAC granulaire des agents et prisons de périmètre fichiers (backend/agent_rbac_jail.py)

Problème : Si chaque sous-agent possède un accès en écriture complet à tout le répertoire de projet, un agent frontal peut écraser par inadvertance des fichiers noyau du backend ou des clés .env.

Mécanique : Lie chaque rôle d'agent à une liste d'outils fixe et à une prison de répertoire autorisée (p. ex. static/ pour les agents frontaux).

Bénéfice : Impose le principe des droits minimaux (Least Privilege) pour chaque processus isolé du système d'exploitation.

Prix et revers (arbitrage) : Empêche qu'un agent administrateur entretienne légitimement des fichiers de configuration hors du dossier de projet ou contrôle directement des modèles .env.

Quand désactiver ? et seuils : N'assouplir que ponctuellement pour des tâches dédiées de maintenance système ; la protection .env devrait toujours rester active en exploitation normale. | Seuils : commutateurs pour prison de chemins, verrou .env et verrou de dossiers système (/etc, /root, /proc).

107. Linter de sortie déterministe et vérificateur de politiques (backend/output_policy_linter.py)

Problème : Les modèles de langage retombent occasionnellement malgré le prompt système dans des tics IA

artificiels ou génèrent des snippets shell risqués.

Mécanique : Scanne le texte généré en temps réel au moyen d'expressions régulières, supprime les ouvertures flagorneuses/relents IA et désamorce les ordres destructeurs.

Bénéfice : Assure précision langagière et sécurité technique sans appel LLM de contrôle supplémentaire.

Prix et revers (arbitrage) : Le filtre destructif peut aussi bloquer d'innocents scripts de nettoyage ou migrations de base voulues quand y figurent des ordres comme DROP ou rm -rf.

Quand désactiver ? et seuils : Désactiver ponctuellement le bloqueur destructif quand des scripts de nettoyage de serveurs ou des migrations SQL de réinitialisation sont créés ciblément. | Seuils : commutateurs pour supprimer les tics IA, bloquer les ordres destructeurs et auto-nettoyage.

108. Reçus d'exécution infalsifiables / quittances SHA-256 (backend/execution_receipt_ledger.py)

Problème : Pour les audits de conformité (EU AI Act / ISO 42001), de simples journaux texte ne suffisent pas car ils peuvent être modifiés après coup.

Mécanique : Chaîne chaque facturation de tâche (modèle, jetons, coûts USD, outils exécutés) avec le hachage SHA-256 de l'entrée précédente dans une chaîne de type blockchain.

Bénéfice : Toute manipulation ultérieure des protocoles de coûts ou de décisions saute aussitôt aux yeux lors du contrôle d'intégrité.

Prix et revers (arbitrage) : Cause un surcoût minimal de calcul et de stockage (~2 ms et ~2 Ko de mémoire SQLite par tâche) ; la chaîne de hachage croît continûment au fil des mois.

Quand désactiver ? et seuils : Peut rester durablement activé en production ; ne désactiver que sur des microsystèmes à mémoire extrêmement limitée. | Seuils : chaînage de hachage (SHA-256) + ventilation détaillée des coûts par exécution.

17. Auto-optimisation, évaluation et intelligence en essaim

109. Optimiseur automatique de prompts selon le principe DSPy/TextGrad (backend/prompt_optimizer.py)

Problème : Des prompts système statiques répètent toujours les mêmes erreurs tant qu'aucun humain ne retouche manuellement.

Mécanique : Évalue les journaux d'erreurs réels d'un rôle, détecte les faiblesses systématiques et génère des règles de garde-fous ciblées pour le prompt système.

Bénéfice : Les agents apprennent continûment des exécutions échouées et durcissent leurs instructions de façon autonome.

Prix et revers (arbitrage) : Si l'auto-apply est actif, une règle fautivement dérivée peut fausser durablement le comportement de l'agent (dérive de prompt) et gonfler le prompt système.

Quand désactiver ? et seuils : Laisser l'auto-apply désactivé en production pour que les patchs de prompts proposés ne soient repris qu'après contrôle humain. | Seuils : à partir d'erreurs : 1 = surréagit aux erreurs isolées | > 8 = apprend très lentement (std : 3). Règles max : > 10 = gonfle le prompt système (std : 5).

110. Suite de tests de régression gold-standard / lanceur d'éval (backend/eval_regression_runner.py)

Problème : Un changement en apparence mineur du prompt système ou du modèle peut dégrader inaperçu des règles de sécurité ou de format existantes.

Mécanique : Exécute avant la reprise de nouveaux réglages des cas de test gold-standard automatisés et contrôle le taux de réussite ainsi que les modèles interdits.

Bénéfice : Prévient les régressions de qualité en production par un portail qualité objectif.

Prix et revers (arbitrage) : Chaque cycle d'éval complet tire tous les prompts de test enregistrés contre l'API LLM (coûte selon l'ampleur des tests ~5 000–25 000 jetons et 15–60 s d'exécution).

Quand désactiver ? et seuils : Désactiver le blocage de déploiement quand des tests de prompts expérimentaux rapides doivent être menés sans attente. | Seuils : taux de réussite min. : < 75 % = laisse passer les dégradations | 100 % = bloque déjà aux moindres écarts (std : 90 %).

111. Routeur de modèles à bandit multi-bras (backend/bandit_model_router.py)

Problème : Des affectations de modèles fixes ne réagissent pas quand un fournisseur cloud devient soudain plus lent ou montre des taux d'erreur accrus.

Mécanique : Note chaque modèle en continu selon le taux de succès réel (60 %) et la vitesse (40 %) et choisit dynamiquement le meilleur candidat.

Bénéfice : Maximise automatiquement qualité et vitesse de réponse à partir de la télémétrie réelle d'exécution.

Prix et revers (arbitrage) : Via le taux d'exploration (epsilon, p. ex. 0.15 = 15 %), le routeur teste délibérément toujours à nouveau des modèles alternatifs qui peuvent dans le cas isolé être plus lents.

Quand désactiver ? et seuils : Désactiver (ou mettre epsilon à 0.00) quand un modèle 100 % fixe et prévisible doit être imposé pour chaque exécution. | Seuils : epsilon : 0.00 = utilise obstinément le meilleur modèle | > 0.30 = essaie trop souvent des modèles plus faibles (std : 0.15). Poids succès/latence : std 0.60 / 0.40.

112. Génération dynamique d'équipes éphémères (backend/ephemeral_team_spawner.py)

Problème : Un agent généraliste isolé est vite dépassé par de grandes tâches transversales (architecture + UI + sécurité + QA).

Mécanique : Recrute selon le briefing de mission une task-force temporaire de spécialistes et la dissout après la mission accomplie.

Bénéfice : Regroupe le savoir spécialisé selon le besoin sans surcoût durable de ressources.

Prix et revers (arbitrage) : Une équipe ad hoc de 4 têtes multiplie la consommation de jetons (3x à 5x) et requiert un surcoût de coordination entre les membres.

Quand désactiver ? et seuils : Désactiver pour les tâches isolées simples ou un budget de jetons restreint ; limiter la taille d'équipe max au quotidien à 3–4. | Seuils : taille d'équipe max : 2–4 = idéal | > 5 = fort surcoût de communication et 5x consommation de jetons (std : 4).

113. Protecteur de ressources en direct et contre-pression hôte (backend/resource_backpressure.py)

Problème : Conteneurs parallèles, audits navigateur et processus locaux peuvent lors de pics de charge surcharger la RAM hôte ou le CPU (OOM-kill).

Mécanique : Surveille les métriques CPU, RAM et disque en temps réel et bride automatiquement les tâches de fond faiblement priorisées dès 85 % d'utilisation.

Bénéfice : Protège le serveur de façon fiable contre les plantages par surcharge et maintient les processus centraux critiques toujours réactifs.

Prix et revers (arbitrage) : Si les seuils sont réglés trop bas (CPU < 70 % ou RAM < 70 %), les tâches de fond sont constamment freinées dès les brèves pointes normales.

Quand désactiver ? et seuils : Ne jamais désactiver complètement — relever plutôt les seuils CPU/RAM à 90 % sur du matériel puissant. | Seuils : Max CPU/RAM : < 70 % = bride au travail normal | > 92 % = risque OOM (std : 85 %). Disque libre min. : std 2.0 Go.

18. Pilier 3 : économie et ERP (alpha) et suite de mandants

Module / vue	Étendue fonctionnelle et caractéristiques techniques (v0.5.1)
Adresses et CRM + sync PocketBase	Base centrale de contacts, débiteurs (10000+) et créanciers (70000+), tiroir de contact 360°, chronologie multi-canal, calendrier/relances ainsi que sync LinkedIn et médias via PocketBase (ports 3005/8090/8092).
Articles et prestations (ART)	Catalogue de données de base dès ART-1001 pour services, taux horaires, forfaits, abonnements SaaS ainsi que produits physiques avec gestion des stocks.
Affaires et contrats	Chaîne continue de pièces offre (ANG-YYYY-NNNN) → commande (AB-YYYY-NNNN), y compris pipeline d'affaires pondéré, prévision de chiffre d'affaires et radar d'échéances contractuelles (alertes 30/14/7 jours).

Module / vue	Étendue fonctionnelle et caractéristiques techniques (v0.5.1)
Comptabilité et DATEV	Pièces sortantes et entrantes selon § 14 UStG, QR de paiement GiroCode, analytique financière, relances automatiques (moteur de recouvrement) et export EXTF/CSV pour DATEV.
Sociétés et isolation des mandants	Plusieurs entreprises propres (company_id) avec bases SQLite physiquement séparées en mode WAL, cycles de numérotation propres, papiers à en-tête et éditeur de modèles.

19. Fonctionnalités d'architecture, de sécurité et de déploiement (v0.5.0 – v0.5.1)

Fonctionnalité / sous-système	Implémentation et bénéfice d'exploitation (v0.5.1)
Architecture de routeurs modulaires	Externalisation vers des routeurs FastAPI dédiés sous archon/routers/ (main.py réduit de 16 254 à 1 875 lignes, -88 %) et couche de configuration centrale archon/core/config.py.
Durcissement sécurité (Archon Hardened)	Clôture de 25 constats d'audit critiques et 68 élevés ; délestage du pool de threads Starlette, chemins DOM UI durcis et proxy d'admin Docker de classe A pour hermes_updater.
Gestion d'utilisateurs RBAC (17.10)	Rôles granulaires (admin, operator, viewer) via backend/auth_service.py avec hachage de mots de passe PBKDF2-HMAC-SHA256 à 100 000 itérations et mode mono-utilisateur/multi-utilisateur.
Mise à jour système en 2 phases (17.11)	Affichage immédiat de la version/HEAD locale via ?local_only=true (~0,25 s) plus rapprochement GitHub asynchrone, instantanés SQLite en ligne pré-update et rollback en 1 clic.
Proxy AnythingLLM et pile Chroma	Intégration transparente d'AnythingLLM dans la bibliothèque centrale via /anythingllm-proxy, y compris fast-path /assets/* et index vectoriel ChromaDB local.
Installateur portable 1-Click	Première installation multi-plateforme via setup.bat, setup.ps1 et install.sh avec docker-compose.yml, .env.example, INSTALL_AGENT.md et doc GitHub en 9 langues.
Portail qualité à 4 yeux et bac à sable de code	Contre-contrôle automatique dans le Kanban avant clôture de ticket ainsi que bac à sable de code interactif dans le Showroom (4.4.1) pour l'aperçu en direct des artefacts générés.

20. Glossaire alphabétique et index

Index alphabétique de tous les modules centraux, composants d'architecture et fonctionnalités (v0.5.1) avec numéros de page exacts.

Fonctionnalité / terme	Brève description	Page(s)
Anti-AI-Slop et STE100	Filtres de style et de mots pour éviter les formules inutiles et économiser des jetons de sortie.	—
Architecture de routeurs (archon/routers)	Répartition modulaire du backend en routeurs FastAPI élancés (-88 % de taille de fichier dans main.py).	p. 32
Archon Hardened (v0.5.1)	Release de sécurité et de stabilité avec structure de routeurs modulaire et constats d'audit clôturés.	p. 3, 32
Bac à sable de code éphémère	Runtime conteneurisé volatil et isolé pour code Python et scripts sûr.	—
Chaîne de pièces ERP (ANG -> AB -> RE)	Documents commerciaux et comptables liés, y compris prévision de pipeline et relances.	—
Chien de garde et superviseur de deadlocks	Supervision à 30 secondes des tâches pendantes, des conteneurs et de l'auto-guérison.	—
ChromaDB	Base vectorielle locale pour la récupération documentaire sémantique dans le pilier 2.	p. 32
Clés d'idempotence et essai à blanc	Protection contre la double exécution d'outils ainsi que mode simulation sans accès en écriture.	—
Compaction de contexte et LRU-K	Décharge mémoire et compression des longs contextes de chat à l'atteinte de la limite RAM.	—
Consolidation du cycle de sommeil	Déduplication nocturne et condensation à long terme de la mémoire des agents.	—
Contrats SLA (application des contrats)	Assurances formelles de transfert et de qualité entre agents spécialisés.	p. 24, 25
Decision Engine (55 frameworks)	Module de stratégie pour débats Boardroom structurés sur 9 catégories méthodologiques.	—
Estimateur de coûts pré-vol	Calcul préalable des coûts en jetons attendus avec arrêt du budget de tâche en cas de dépassement.	p. 10
Export DATEV et GiroCode	Interface EXTF/CSV pour données comptables et QR de paiement EPC sur factures.	—
Exécution durable et Time-Travel	Longues exécutions interruptibles avec checkpoints SQLite et replay d'exécution pas à pas.	—
File de lettres mortes (DLQ)	File d'erreurs avec pilotage automatique de répétition pour tâches échouées.	p. 28
Graphe de connaissances	Magasin local de triplets pour relations d'entités structurées à côté des données vectorielles.	p. 17, 26
Journal d'audit (chaîne de hachage SHA-256)	Chaîne de protocoles révisable de toutes les actions système et d'agents.	p. 18
Maille événementielle et déclencheurs webhook	Traitement événementiel des webhooks entrants et modifications de dossiers.	—
MCP (Model Context Protocol)	Architecture de pilotes standardisée pour une connexion sûre des outils.	p. 6, 14

Fonctionnalité / terme	Brève description	Page(s)
Mise en cache sémantique	Stockage intermédiaire de 24h de prompts similaires en contenu pour économiser des jetons.	p. 25
Nettoyeur PII et réhydratation	Anonymisation bidirectionnelle des données personnelles sensibles avant appels LLM externes.	—
Optimiseur DSPy / TextGrad	Amélioration automatique des prompts système à partir de métriques d'évaluation réelles.	—
Orchestrateur maître Hermes	Moteur central pour décomposition d'objectifs, délégation et synthèse dans le démon sur le port 8420.	p. 11
Ordonnanceur DAG	Planification d'exécution à deux niveaux des sous-tâches dépendantes avec fan-out / fan-in parallèle.	p. 13
Pare-feu sortant	Contrôle réseau et domaines sortants pour les appels d'outils des agents.	p. 29
Passerelle IA OmniRoute	Passerelle multi-fournisseurs incluant routage niveau gratuit, couverture de latence et arène d'éval.	—
PBKDF2 / Argon2id RBAC	Sécurisation des rôles et mots de passe dans le System Control Center (onglet 17.10).	—
Plugins Evey / Hermes	Extensions système modulaires pour mémoire, délégation, vérification et interfaces.	—
Pool de connexions SQLite WAL	Accès base multi-thread sûr en écriture via Write-Ahead Logging.	p. 26
Portail d'approbation (HITL)	Étape de contrôle humain avant e-mails externes, écritures ou accès fichiers critiques.	p. 3
Portail qualité à 4 yeux	Contrôle indépendant des tâches Kanban terminées par un agent réviseur.	p. 3, 32
Proxy AnythingLLM	Intégration locale d'AnythingLLM + ChromaDB via /anythingllm-proxy dans la bibliothèque centrale.	p. 3, 32
Pré-mortem et portail de consensus	Test de résistance d'erreurs simulé et vote multi-agents avant validations critiques.	—
RAG correctif (CRAG)	Interrogation de connaissances auto-vérifiée avec évaluation de pertinence avant génération de réponse.	p. 27
Recherche hybride (FTS5 + RRF)	Combinaison de recherche plein texte BM25 et recherche vectorielle par fusion de rangs réciproques.	p. 3, 26
Research Lab	Recherche web et de sources multi-niveaux avec triangulation des preuves.	p. 3, 7, 21
Routage de coûts à 3 niveaux	Sélection automatique de modèles en LOW / MEDIUM / HIGH selon complexité de tâche et budget.	—
Routeur de modèles Bandit	Routeur adaptatif (bandit multi-bras) pour le choix dynamique de modèles selon taux de réussite et coûts.	—
Routeur JEV (intention 0-jeton)	Préfiltre déterministe sans coûts IA pour ordres courts univoques.	p. 10
Sociétés et capacité multi-mandants	Séparation physique des bases de plusieurs sociétés via company_id.	—

Fonctionnalité / terme	Brève description	Page(s)
System Control Center (11 onglets)	Console d'administration centrale incluant utilisateurs/RBAC et mise à jour système en 2 phases.	p. 8, 25
Token Guard et tueur de boucles	Protection temps réel contre les boucles infinies (contrôle Jaccard) et les dépassements de jetons.	—
Écran bot (navigateur web)	Moniteur d'exécution visuel pour interactions navigateur via Playwright.	—